

Aim: To study MS-DOS commands for performing Character User Interface (CUI) operations in Files and Directories.

Procedure:

1. Go to **START → RUN** and type '**Cmd**' and click ok button. The command prompt window will open.
2. The default directory location is **C:\Documents and Settings\Username>**, now change the prompt like **C:\> D:**
3. Type the following commands in the prompt **D:\>** to see the result

DIRECTORY RELATED COMMANDS:

1. DIR:

This command is used to display the files and directories of the current drive and directory.

Syntax: DIR [drive:][[path]][filename]

DIR /P: To display files and directories page wise.

DIR /W: To display files and directories width wise (horizontally).

DIR /AD: To display directories only.

DIR /AH: To display hidden files only.

DIR /AR: To display read only files.

DIR /ON: To display files in sorted order name wise.

DIR /OS: To display files in sorted order size wise.

Example:

D:\> DIR

D:\> DIR /P

D:\> DIR /W

D:\> DIR /AH

2. MD [MAKE DIRECTORY]:

This command is used to create a new folder or directory.

Syntax: MD [drive:] [path]

Where drive is the floppy \ hard disk drive on which you want to create a new directory.

Example:

```
D:\> MD RGCET
```

```
D:\> DIR /P
```

3. CD [CHANGE DIRECTORY]:

This command is used to change current directory to another directory.

Syntax: CD [drive:][path]

CD.. : This command is used to go to previous directory.

CD : This command is used to root directory.

Example:

```
D:\> CD RGCET
```

```
D:\RGCET>
```

```
D:\> CD..
```

```
D:\>
```

4. RD [REMOVE DIRECTORY]:

This command is used to remove the directory.

Syntax: RD [directory]

Where directory is the name of the directory which we want to remove.

Example:

```
D:\> RD RGCET
```

```
D:\> DIR /P
```

FILE RELATED COMMANDS:

1. COPY CON COMMAND :

This command is used to create a new file.

Syntax : COPY CON [File Name]

Example:

```
D:\> copy CON COLL.TXT
```

Rajiv Gandhi College of Engineering and Technology is a premier private engineering institute located in Puducherry, India under Sri Balaji Educational and charitable Public Trust.

Ctrl Z or F6 – to save the file.

2. TYPE COMMAND:

This command is used to display the contents of the file.

Syntax: TYPE [File Name]

Example: D:\> Type college

3. COPY COMMAND:

This command is used to copy an existing file to a new file.

Syntax: COPY [Existing Filename] [New Filename]

Example: D:\>Copy college newcol

4. REN COMMAND:

This command is used to change the file name.

Syntax: REN [Old File Name] [New Name]

Example:

```
D:\>REN newcol newcollege
```

5. DEL COMMAND:

This command is used to delete the file.

Syntax: DEL [File Name]

Example:

D:\>DEL college

6. CLS COMMAND:

CLS is a command that allows a user to clear the complete contents of the screen and leave only the prompt.

Syntax: CLS

Example:

D:\> CLS

7. TIME COMMAND:

To display the time and to change the current time

Syntax: TIME [time]

Example:

D:\> TIME

8. DATE COMMAND:

To display the date and also to change the Current date.

Syntax: DATE [date]

Example:

D:\> DATE

9. EDIT COMMAND:

This command is used to change or edit the content of the given file.

To Save file - Ctrl+F+S

To Exit file - Alt + X

SCREEN SHOT:

```
D:\RGCET>MD MYDIR
```

```
D:\RGCET>CD MYDIR
```

```
D:\RGCET\MYDIR>COPY CON COLL.TXT
```

```
Rajiv Gandhi College of Engineering and Technology is a premier private engineer  
ing institute located in Puducherry, India under Sri Balaji Educational and Char  
itable Public Trust^Z
```

```
1 file(s) copied.
```

```
D:\RGCET\MYDIR>TIME
```

```
The current time is: 12:14:14.80
```

```
Enter the new time:
```

```
D:\RGCET\MYDIR>TIME
```

```
The current time is: 12:14:24.95
```

```
Enter the new time: 9:33 PM
```

```
D:\RGCET\MYDIR>DATE
```

```
The current date is: Mon 02/17/2014
```

```
Enter the new date: (mm-dd-yy)
```

```
D:\RGCET\MYDIR>IPCONFIG
```

```
Windows IP Configuration
```

```
Ethernet adapter Local Area Connection:
```

```
Media State . . . . . : Media disconnected  
Connection-specific DNS Suffix . :
```

RESULT:

Thus the given MS DOS directory related and file related commands have been studied and executed successfully.

PROGRAM: 1 Study of OS Commands

1. What is a Computer?

A computer is a programmable machine or device that performs pre-defined or programmed computations or controls operations that are expressible in numerical or logical terms at high speed and with great accuracy.

2. What is CPU?

Central Processing Unit. The CPU consists of a *control unit*, an *arithmetic/logic unit (ALU)*, and *registers*.

3. What is ALU?

Arithmetic Logic Unit

- The task of performing operations *like arithmetic and logical operations* is called processing.
- ALU Comprises of two units

Arithmetic unit

Logic unit

4. What is CU?

Control unit- controls the other parts of the computer

5. What is RAM?

Random Access Memory- Volatile

6. What is ROM?

Read only memory – Non-volatile

7. Differentiate between main memory and secondary memory.

Main memory:-

- Fast in operation
- Small Capacity
- Expensive

Secondary memory:-

- Slower than primary storage
- Large Capacity
- Less price

8. What is hardware?

Hardware refers to all visible devices that are assembled together to build a computer system. These include various input devices, central processing unit, output devices and memory.

9. Mention the different types of hardware components?

CPU, Input devices, Output devices, Storage devices

10. What is meant by softcopy output?

Output displayed on the monitor. **Soft copy:** Electronic version of the output is referred as soft copy cannot be preserved

11. What is meant by hardcopy output?

Output printed on the paper. **Hard copy:** Physical form of output is referred to as hard copy. Can be preserved for future reference

12. What is an operating system?

Operating system (OS) is an interface between hardware and user. Operating system is the first layer of software loaded into computer memory.

13. What are language processors?

A language processor is a special type of a computer software that has the capacity of translator the source code or program codes into machine codes. The following are different types of language processors are:

- Compiler
- Assembler
- Interpreter

14. Mention some operating systems.

MSDOS, Windows XP, Windows 7, UNIX, Ubuntu

15. Differentiate between compiler and assembler.

The program written in any high level language is converted into machine language using a compiler. The assembly language program is translated into machine code by assembler.

16. What is translator?

It is a program which translates the program written in a programming language to machine language.

17. What is meant by interpretation?

Interpreter analyses and executes the source code in line-by-line manner , without looking at the entire program.

18. What is meant by source program?

Program written in high level language

19. What is meant by object program?

Program given in machine language

20. Give examples for High Level Languages.

Eg. C , C++, java

21. Differentiate between internal & external DOS commands.

Internal commands : copy, del, rename ;

External commands : sort, date

22. What is the use of copy and del command?

Copy command is used for copying/ duplicating the file. One source file and the other destination or copied file.

23. What is the use of rename command?

This command is used for giving a new name for an existing file

24. Give the syntax of change directory

C:>cd foldername <enter>

25. What is the use of directory? How to create a new directory?

Directory is a folder / storage area where multiple files are stored/kept.

C:>Md folder name<Enter>

26. Define system software.

It consists of variety of programs that supports the operation of the computer.

This software makes it possible for the user to focus on the other problems to be

solved without needing to know how the machine works internally. Eg: operating system, assembler, loader.

27. Give some applications of operating system.

Operating system acts as an interface between the user and the system. Eg: windows, linux, unix, dos

- to make the computer easier to use
- to manage the resources in computer
- process management
- data and memory management
- to provide security to the user.

28. Define compiler and interpreter.

Compiler is a set of programs which converts the whole high level language program to machine language program.

Interpreter is a set of programs which converts high level language program to machine language program line by line

29. Differentiate between LAN and WAN.

LAN	WAN
Local Area Network	Wide Area Network
LAN is an interconnected collection of computers placed within a small area	<i>WAN is an interconnected collection of computers placed at large distances.</i>
The connection between computers are by means of LAN cables	The connection between computers are by means of telephone cables or by satellite links.

30. Define Internet

Internet is defined as the network of networks. Internet is a network which covers the whole world. To this millions of computers are connected.

31. What is the Difference between Internet and Intranet?

Similarities in Internet & Intranet

- Intranet uses the internet protocols such as TCP/IP and FTP.

- Intranet sites are accessible via web browser in similar way as websites in internet. But only members of Intranet network can access intranet hosted sites.
- In Intranet, own instant messengers can be used as similar to yahoo messenger/gtalk over the internet.

Differences in Internet & Intranet

- Internet is general to PCs all over the world where Intranet is specific to few PCs.
- Internet is wider access and provides a better access to websites to large population whereas Intranet is restricted.
- Internet is not as safe as Intranet as Intranet can be safely privatized as per the need.

EX:NO:2	AREA OF A TRIANGLE	DATE:
---------	--------------------	-------

AIM :

To write a C program to find the area of a triangle.

Formula of area of any triangle:

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$$

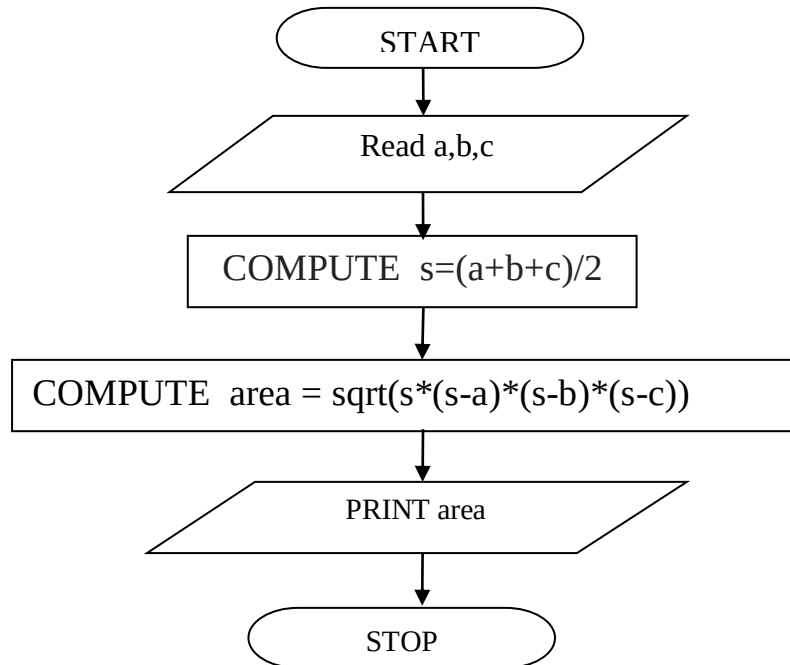
Where $s = (a + b + c)/2$

ALGORITHM:

- Step 1 : start
- Step 2 : Declare the variable a, b, c, s, area
- Step 3 : Get the value of a, b, c

Step 4 : Find s value $s=(a+b+c)/2$
Step 5 : Find area = $\text{sqrt}(s*(s-a)*(s-b)*(s-c))$
Step 6 : Print the value of area
Step 7 : Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.2 Area of Triangle
#include<stdio.h>
#include<math.h>
#include<conio.h>
void main()
{
    clrscr();
    float a,b,c;
    float s,area;
    printf("\nAREA OF THE TRIANGLE");
    printf("\n*-----*");
    printf("\nEnter size of each sides of triangle");
    printf("\n*-----*");
    printf("\nEnter 1st side size : "); scanf("%f",&a);
```

```

printf("\nEnter 2nd side size : "); scanf("%f",&b);
printf("\nEnter 3rd side size : "); scanf("%f",&c);
s = (a+b+c)/f;
area = sqrt(s*(s-a)*(s-b)*(s-c));
printf("Area of triangle is: %f",area);
getch();
}

```

OUTPUT :-

```

AREA OF THE TRIANGLE
*-----*
Enter size of each sides of triangle
*-----*
Enter 1st side size : 5

Enter 2nd side size : 6

Enter 3rd side size : 7

Area of triangle is: 14.70

```

RESULT:

Thus the given program to calculate area of a triangle has been executed and verified successfully.

PROGRAM 2: Area of Triangle

1. What is a program?

A **program** is a **set** instruction written to carryout a particular task, so that Computer can perform some specified task.

2. What is algorithm?

Algorithm means the logic of a program. It is a step-by-step description of how to arrive at a solution of a given problem.

3. What are the steps to solve the problem in a computer system?

- Problem must be analyzed thoroughly.
- Solution method is broken down into a sequence of small tasks.
- Based on this analysis, an algorithm must be prepared to solve the problem.
- The algorithm is expressed in a precise notation. This notation is known as "Computer Program".
- The Computer program is fed to the computer.
- The instruction in the program executes one after another and outputs the expected result.

4. What is BCPL?

Basic Combined Programming Language

5. Who developed BCPL?

Short for **Basic Combined Programming Language**, **BCPL** is a programming language developed in 1966 by Martin Richards of the University of Cambridge that became a popular because of its portability.

6. Who developed B language?

B language developed in the year **1960** at Cambridge university.

7. Who developed C language?

developed by **Dennis Ritchie at Bell Laboratories in 1972.**

8. How do you make comments in C program?

By using `/* ---*/` or by `//`

9. What is preprocessor statement?

Before compilation some instructions are given to the compiler by the preprocessor directives .

Eg. `#include<stdio.h>`

10. Differentiate between constant and variable.

CONSTANTS : Constants which do not change during the execution of a program.

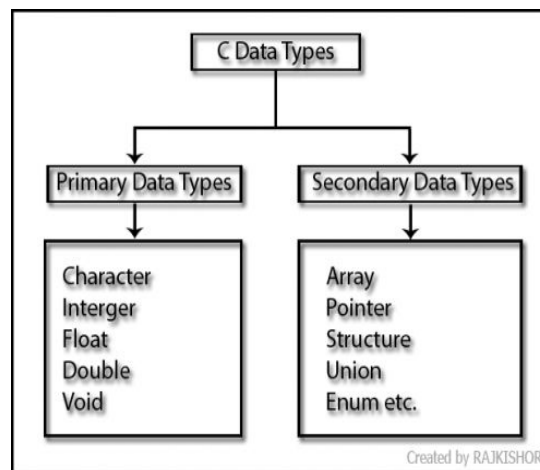
There are several types of constants in C.

C supports several types of constants.

VARIABLES: A variable is a value that can change any time. It is a memory location used to store a data value. Variables can be of different data types. The data types are integers, real or character constants. A variable is a data name used for storing a data values.

11. What is data type?

Data types: a program usually contains different types of data types (integer, float, character etc.) To store the values being used in the program along with some library function and user defined function (udf) to process that stored



Name the basic data types of C.

There are four basic data types available in 'C'.

int

float

char

double

12. Differentiate between string constant and character constant.

String constant is a group of characters enclosed within double quotation mark.

Character constant consists of single character enclosed within single quotation mark

13. What is the range of integer, char, float for a 16-bit computer?

int - 2 Bytes

char - 1 Byte

float - 4 Bytes

long int - 4 Bytes

double - 8 Bytes

14. What is a statement?

Each 'C' statement performs a single operation and ends with semicolon.

Eg a= 5; assignment statement

15. What is a keyword?

Keywords are certain reserved words that have standard and pre-defined meaning in 'C'. These keywords can be used only for their intended purpose.

16. Differentiate between keywords and identifiers.

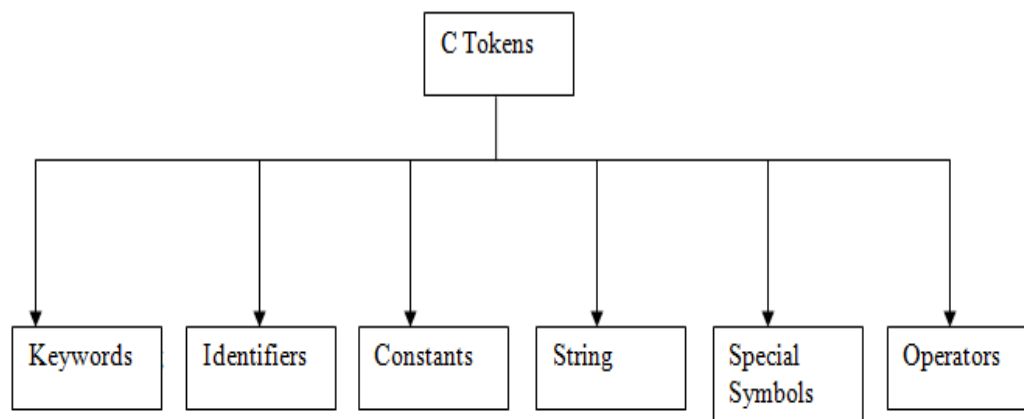
IDENTIFIERS

Identifiers are names of variables, functions and array.

The (_) underscore symbol can be used as an identifiers

Eg: # define N 10, int A

N and **A** are user defined identifiers



EX:NO:3	STUDENT MARKLIST	DATE:
---------	------------------	-------

AIM :

To write a C program to find the total and average percentage obtained by a student for 6 subjects.

ALGORITHM:

Step 1 : Start

Step 2 : Declare the variable m1, m2, m3, m4, m5, m6, total, avg, percentage

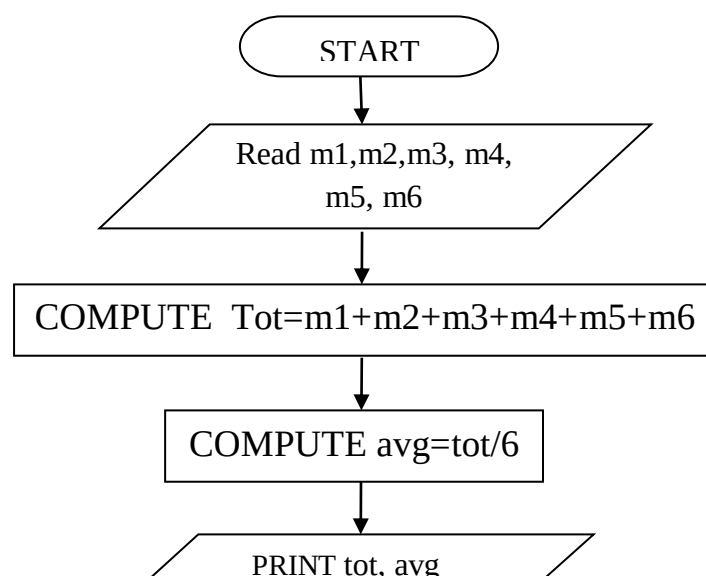
Step 3 : Get the values of m1, m2, m3, m4, m5, m6

Step 4 : Calculate the total and average values

Step 5 : Print the total and average values

Step 6 : Stop

FLOW CHART :



SOURCE CODE :

//Program No.3 C Program for STUDENT MARKLIST.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
int roll,m1,m2,m3,m4,m5,m6,tot;
```

```
float avg;
```

```
char sname[20];
```

```
printf("\nA STUDENT MARKLIST");
```

```
printf("\n*-----*");
```

```
printf("\nEnter information of students:\n\n");
```

```
printf("Enter name: ");
```

```
scanf("%s",sname);
```

```
printf("Enter roll number: ");
```

```
scanf("%d",&roll);
```

```
printf("\nEnter 6 diffenent marks: ");
```

```
printf("\nEnter ES Mark : ");scanf("%d",&m1);
```

```
printf("Enter TD Mark : ");scanf("%d",&m2);
```

```
printf("Enter CP Mark : ");scanf("%d",&m3);
```

```
printf("Enter EEE Mark: ");scanf("%d",&m4);
```

```
printf("Enter ECE Mark: ");scanf("%d",&m5);
```

```
printf("Enter MS Mark : ");scanf("%d",&m6);
```

```
tot=m1+m2+m3+m4+m5+m6;
```

```
avg=tot / 6;
```

```
printf("\nDisplaying Information");
```

```
printf("\n*-*-*-*-*-*-*-*-*-*");
```

$$\}$$

SCREEN SHOT:

OUTPUT :

A STUDENT MARKLIST

* _ _ _ _ _ *

Enter information of students:

Enter name: Vijaykrishna

Enter roll number: 256

Enter 6 diffenent marks:

Enter ES Mark : 67

Enter TD Mark : 89

Enter CP Mark : 69

Enter EEE Mark: 56

Enter ECE Mark: 99

Enter MS Mark : 71

Displaying Information

*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*

Student Name: Vijaykrishna

Student Roll: 256

-----Marks Obtained-----

ES TD CP EEE ECE MS

67 89 69 56 99 71

Total: 451

Average:75.00

RESULT:

Thus the given program to calculate the total and average percentage has been executed and verified successfully.

PROGRAM 3: Total and Average Calculation

1.What is the need for an escape sequences?

\n	New line
\b	Backspace
\f	Form feed
\'	Single quote
\\	Backslash
\0	Null

2. What is a symbolic constant?

CONSTANTS

Constants which do not change during the execution of a program. There are several types of constants in C. C supports several types of constants.

3. How do you classify C operators?

Types of C operators:

C language offers many types of operators. They are,

- Arithmetic operators
- Increment/decrement operators
- Assignment operators
- Relational operators
- Logical operators
- Conditional operators (ternary operators)
- Bit wise operators
- Special operators

4. What are relational operators?

Relational operators are used to find the relation between two variables. i.e. to compare the values of two variables in a C program.

<i>S.no</i>	<i>Operators</i>	<i>Description</i>	<i>Example x=5,y=7</i>
1	>	x is greater than y	If(x>y) then return false(0)
2	<	x is less than y	If(x<y) then return true(1)
3	>=	x is greater than or equal to y	If(x>=y) then return false(0)
4	<=	x is less than or equal to y	If(x<=y) then return true(1)
5	==	x is equal to y	If(x==y) then return false(0)
6	!=	x is not equal to y	If(x!=y) then return true(1)

5. What is unary operator?

An unary operator is used for specifying positive value or negative value for a variable or expression

Eg. -5,+a,-b;

6. WHAT IS BINARY OPERATOR?

Binary operators are used for performing binary operations namely addition, subtraction, multiplication and division.

7. What is typecasting?

Type casting is the process of converting the value of an expression to a particular data type.

Example:

int x,y;

c = (float) x/y; where a and y are defined as integers. Then the result of x/y is converted into float.

8. What is a conditional / ternary operator?

Conditional operators return one value if condition is true and returns another value if condition is false. This operator is also called as ternary operator.

Syntax : (Condition? true_value: false_value);

Example : (A > 100 ? 0 : 1);

In above example, if A is greater than 100, 0 is returned else 1 is returned. This is equal to if else conditional statements.

9. Differentiate between && and &.

&& logical AND (x>5) &&(y<5) It returns true when both conditions are true

& this is used to get the address of the variable. Example: & will give address of a.

10. Differentiate between pre-increment/decrement & post-increment/decrement.

Pre increment	++i	Value of i is incremented first then incremented is assigned to variable i.
Post-increment	i++	Value is assigned to the variable first and then assigned value get incremented.
Pre decrement	--i	Value of i is decremented first then incremented is assigned to variable i.
Post decrement	i--	Value is assigned to the variable first and then assigned value get decremented.

11. What is an expression ?

An expression is a combination of operands and operators. Eg. a+b This is an arithmetic expression

12. What are Relational Operators?

Relational operators are used to find the relation between two variables. i.e. to compare the values of two variables in a C program.

<i>S.no</i>	<i>Operators</i>	<i>Description</i>	<i>Example x=5,y=7</i>
1	>	x is greater than y	If(x>y) then return false(0)
2	<	x is less than y	If(x<y) then return true(1)
3	>=	x is greater than or equal to y	If(x>=y) then return false(0)
4	<=	x is less than or equal to y	If(x<=y) then return true(1)
5	==	x is equal to y	If(x==y) then return false(0)
6	!=	x is not equal to y	If(x!=y) then return true(1)

13. What is arithmetic expression ? Give eg.

An expression which uses arithmetic operators to perform arithmetic operation.

Eg. S = a+b

14. What is relational expression ? Give eg.

An Expression which uses relational operators to check relation between variables.

Eg. a>b

15. What is logical expression ?. Give eg.

An Expression which uses logical operators to combine two expressions

Eg. $(a > b) \ \&\& \ (b > c)$

16. What is the use of assignment operator ?

= is the assignment operator. This operator is used for assigning a value to a variable

Eg. `sum = 25;`

17. What is the difference between declaring and defining a variable?

1. Declaring a variable means specifying the data type of the variable. Eg `int a,b;` This statement declares that a and b are two integer variables.
2. Defining a variable means assigning some initial value to the variable Eg. `int a = 6;`
3. This statement is used to assign the value 6 to the variable a.

18. What is an formatted input statement?

The formatted input functions read all types of data values. They require conversion symbol to identify the data type.

Eg `int a; scanf("%d",&a);`

This statement is used to read integer value in the integer variable 'a'

19. What is an formatted output statement?

The formatted output functions output all types of data values. They require conversion symbol to identify the data type.

Eg. `int a; printf("The given number is = %d",a);`

20. What is the difference between `getc()` and `putc()`?

`Getc` is used for getting a char from the user. `Putc()` is used for displaying a char

EX:NO:4	PLACES OF A INDIVIDUAL NUMBER IN A DIGIT	DATE:
---------	--	-------

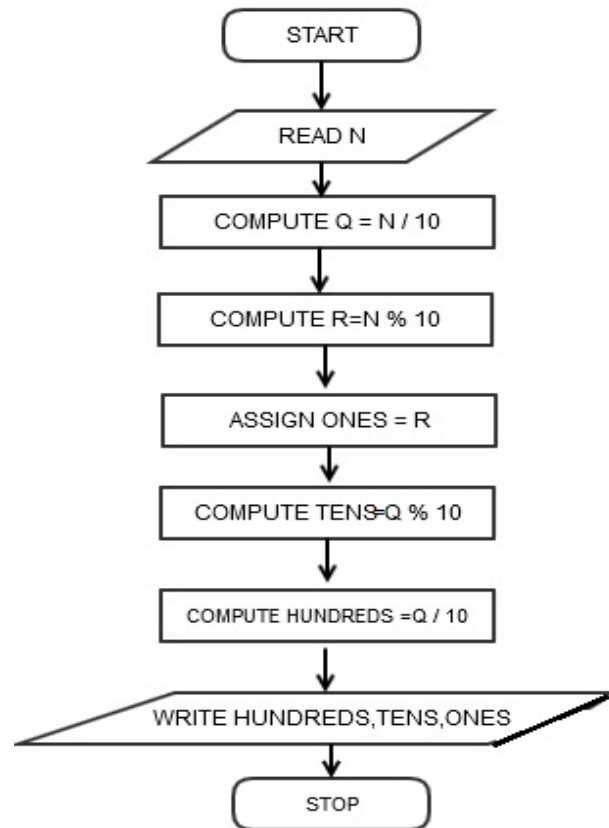
AIM :

To write a C program to read a three digit number and produce output like hundreds, tens, units.

ALGORITHM:

- Step 1 : Start
- Step 2 : Declare the variables n,q,r,units,tens,huns;
- Step 3 : Read the n value
- Step 4 : Find the value of $q = n/10$ and $r = n \% 10$
- Step 5 : Find the value of $units = r$, $tens = q \% 10$, $huns = q / 10$
- Step 6 : Print the values in hundreds, tens and units.
- Step 9 : Stop

FLOW CHART :



SOURCE CODE :

```

//Program No.4 Places of number in a digit.
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    int n,q,r,ones,tens,hundreds;
    printf("\nPLACES OF DIGIT IN A NUMBER");
    printf("\n*-*-*-*-*-*-*-*-*-*-*");
    printf("\nEnter any three digit number: ");
    scanf("%d",&n);
    q=n/10;
  
```

```

r=n%10;
ones=r;
tens=q%10;
hundreds=q/10;
printf("\nYOU ENTERED %d WHICH IS",n);
printf("\n-----");
printf("\n\t %d Hundreds \n\t %d Tens \n\t %d
Ones",hundreds,tens,ones);
printf("\n-----");
getch();
}

```

OUTPUT:

PLACES OF DIGIT IN A NUMBER

*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

Enter any three digit number: 678

YOU ENTERED 678 WHICH IS

6 Hundreds
7 Tens
8 Units

RESULT:

Thus the given C program to read a three digit number and produce output like hundreds, tens, units has been executed and verified successfully.

PROGRAM: 4 Place value of 3 digit number in words

1. What is the use of modulus operator?

'%' is modulo operator. It gives the remainder of an integer division

Example:

a=17, b=6. Then c=%b gives 5.

2. What is meant by mixed mode operation?

If operands in an expression contain both INTEGER and REAL constants or variables, this is a mixed mode arithmetic expression.

In mixed mode arithmetic expressions, INTEGER operands are always converted to REAL before carrying out any computations. As a result, the result of a mixed mode expression is of REAL type. The following is a table showing this fact.

Operator	INTEGER	REAL
INTEGER	INTEGER	REAL
REAL	REAL	REAL

The rules for evaluating mixed mode arithmetic expressions are simple:

Use the rules for evaluating single mode arithmetic expressions for scanning.

After locating an operator for evaluation, do the following:

if the operands of this operator are of the same type, compute the result of this operator.

otherwise, one of the operand is an integer while the other is a real number. In this case, convert the integer to a real (i.e., adding .0 at the end of the integer operand) and compute the result. Note that since both operands are real numbers, the result is a real number.

There is an exception, though. In $a**n$, where a is a real and n is a positive integer, the result is computed by multiplying n copies of a . For example, $3.5**3$ is computed as $3.5*3.5*3.5$

Simple Examples:

- $1 + 2.5$ is 3.5
- $1/2.0$ is 0.5
- $2.0/8$ is 0.25
- $-3**2.0$ is -9.0
- $4.0**(1/2)$ is first converted to $4.0**0$ since $1/2$ is a single mode expression

whose result is 0. Then, $4.0^{**}0$ is 1.0

3. Differentiate between Unformatted and formatted i/o statements.

- Formatted functions: The formatted i/o functions read and write all types of data values. They require conversion symbol to identify the data type.
- Unformatted functions: The unformatted input/output functions only work with the character data type. They do not require conversion symbol for identification of data types because, they work only with character data type.

4. Explain the meaning of Lvalue?

Lvalue or location value is an address that is stored . In the assignment statement, `a=10;`

we have the lvalue of a used on the left hand side and the integer constant value 10 assigned to this location.

In the assignment statement, `b=a;` we have the rvalue of a (i.e. 10) used to assign to the lvalue of b. So depending on which side of an assignment, we use the variable a, we get two different behaviors.

5. What is the need for input-output functions and list its types?

C is a functional programming language. Therefore, it has no exclusive built-in functions to perform the basic input-output operations. To perform these input-output operations C provides a library of functions. This library is called a standard input-output library. It is denoted by `stdio`. The header file containing such library functions is called **`stdio.h`**

There are two types of input-output (I/O) function. They are:

Formatted I/O functions

- `printf ()`
- `scanf ()`

Unformatted I/O functions

- `getchar ()`
- `putchar ()`
- `gets ()`

- puts ()

6. What value is returned when the scanf () function is executed?

The scanf () function returns an int value which specifies the number of input fields successfully scanned, converted and stored.

7. What value is returned when the printf() function is executed?

The printf () returns the number of bytes output to stdout.

8. What is the difference between getche () and getch ()?

The getche () function gets a character from the keyboard and echoes to the screen.

Whereas, the getch () function gets a character from the keyboard and does not echo to the screen.

9. Difference between scanf and printf function.

Scanf	Printf
Used to accept data	Used to display data
Control string and & operator is used	Control string only used.
It end with semicolon	It end with semicolon
Number of input specified with format String and enclosed within double quotes.	Number of input specified with format string and separated by commas.
The Input variables are specified using address operator (&) is separated by commas.	The output variables are specified by their name and separated by commas.

EX:NO:5	VOWEL OR NOT	DATE:
----------------	---------------------	--------------

AIM :

To write a C program to check whether a given character is vowel or not using Switch – Case statement.

ALGORITHM:

Step 1 : Start

Step 2 : Declare the character variable ch

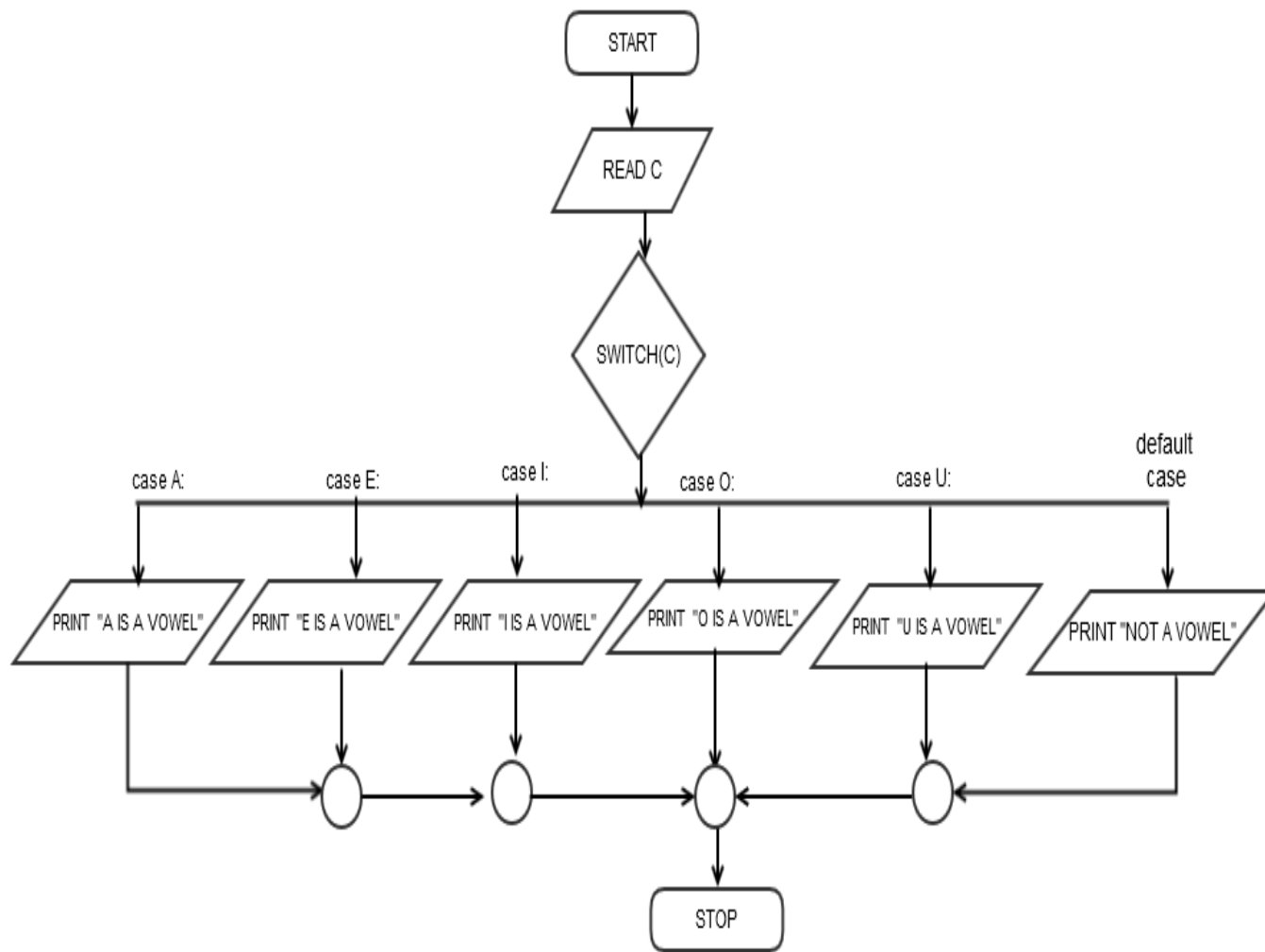
Step 3 : Read the character

Step 4 : i) Using switch case check the character which is suitable for the given value

ii) if no cases matched, then print the default statement.

Step 5 : Stop

FLOW CHART :



SOURCE CODE :

//Program No.5 Vowel checking

```
#include <stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
char ch;
```

```
printf("\nVOWEL OR NOT");
```

```
printf("\n*-----*");
```

```
printf("\nEnter a character : ");
```

```

scanf("%c", &ch);
switch(ch)
{
    case 'a': case 'A':
    case 'e': case 'E':
    case 'i': case 'I':
    case 'o': case 'O':
    case 'u': case 'U':
        printf("%c is a vowel.\n", ch);
        break;
    default:
        printf("%c is not a vowel.\n", ch);
}
getch();
}

```

OUTPUT:

```

VOWEL OR NOT
*-----*
Enter a character : m
m is not a vowel.

```

```

VOWEL OR NOT
*-----*
Enter a character : 0
0 is a vowel.

```

RESULT:

Thus the given C program to check whether a given character is vowel or not using Switch – Case statement has been executed and verified successfully.

EX:NO:5 VOWEL OR NOT

1. What is sequential logic?

Sequence logic is used for performing instructions one after another in a sequence

2. What is iteration logic ?

Iteration logic is used for producing loops in program logic when one or more instructions may be executed several times depending on some condition. It uses two control structures called do...while, and repeat...until.

3. What is selction logic ?

Selection logic is used for selecting the process path out of two or more alternative paths in the program logic. It uses three control structures called if...then, if...then...else and switch...case.

4. What is the use of switch statement ?

It is used for implementing Selection logic. To select one out of the available alternatives.

5. Give the syntax of switch statement ?

```
Switch(exp)
{ case 1:{
    ----
    Break;
}
case 2: :{
    ----
    Break;
}
,
,
Default :
{
    ----
    Break;
}
}
```

6. How do you classify the control statements?

THREE TYPES OF CONTROL STATEMENTS ARE:

- a. Selection or branching
- b. Repetition or looping
- c. Jump or unconditional branching

7. Differentiate between break and continue.

The break statement is used to exit from a loop. It is given in unconditioned loop. It is used within a for loop, while loop and do while loop.

SYNTAX

```
break;
```

The continue statement is used to skip the current iteration of the loop and start the next iteration of the loop. It is used within a for loop, while loop and do while loop

SYNTAX

Continue;

8. What is a char data type?

Character denotes any alphabet, any digit, or any special character enclosed within single quotation mark. Eg. char s='y';

9. What is a string ?

String is a group of characters enclosed within double quotation mark. Eg. "good day"

10. How you will declare a character array?

Char name[20]. Ie 20 locations are reserved for storing the given string.

11. Give the syntax for getting names of 10 students.

```
Char sname[10][15];
```

12. Give the syntax of displaying a name

```
Printf("the given name is : %s",name);
```

EX:NO:6	SQUARE VALUES FOR GIVEN RANGE	DATE:
----------------	--------------------------------------	--------------

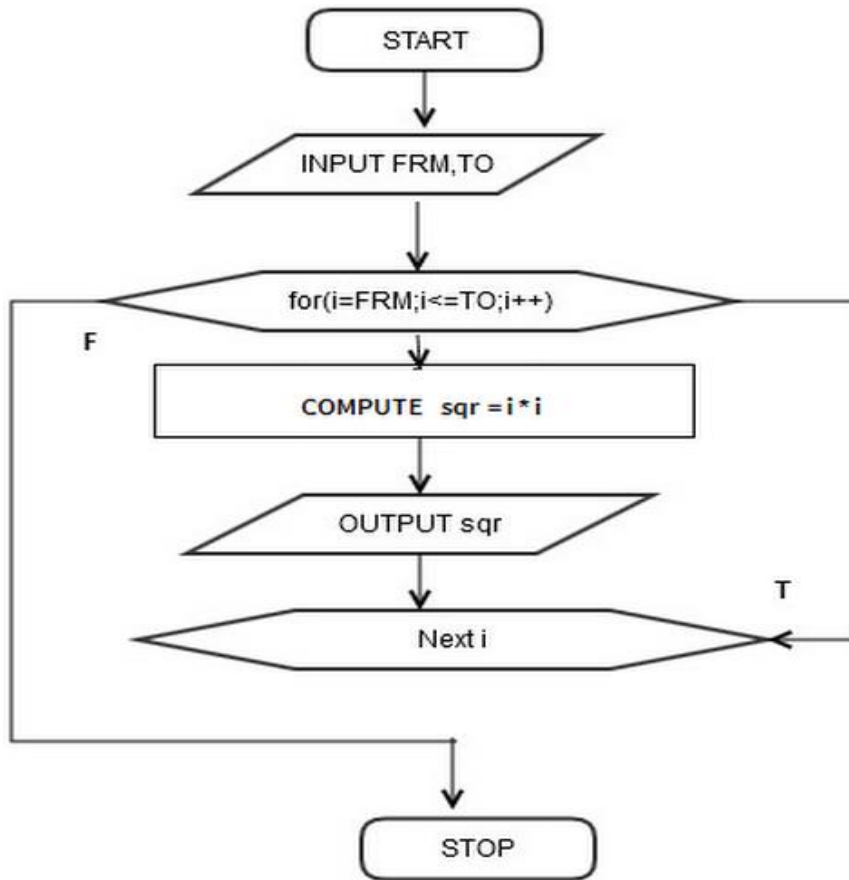
AIM :

To write a C program to print the numbers from 1 to 10 along with their squares.

ALGORITHM:

- Step 1 : Start
- Step 2 : Read starting and ending Number
- Step 3 : Using for loop, calculate $sqr = i * i$ and print the squares values of each number within the range.
- Step 4 : Stop

FLOW CHART :



SOURCE CODE :

//Program No.6 Squares of the series

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
int frm,to,i,sqr;
```

```
printf("\nSQUARES OF THE SERIES");
```

```
printf("\n*-----*");
```

```
printf("\nEnter Starting Range:");
```

```
scanf("%d",&frm);
```

```
printf("Enter Ending Range:");
```

```
scanf("%d",&to);
```

```
for(i=frm;i<=to;i++)
```

```
{
```

```

        sqr=i*i;
        printf("Square of %d is :%d\n",i,sqr);
    }
    gtech();
}

```

OUTPUT :

```

SQUARES OF THE SERIES
*-----*
Enter Starting Range:7

Enter Ending Range:15

Square of 7 is : 49
Square of 8 is : 64
Square of 9 is : 81
Square of 10 is : 100
Square of 11 is : 121
Square of 12 is : 144
Square of 13 is : 169
Square of 14 is : 196
Square of 15 is : 225

```

RESULT:

Thus the given C program to print the numbers from 1 to 10 along with their squares has been executed and verified successfully.

EX: NO: 6 SQUARE VALUES FOR GIVEN RANGE

1. How do you classify the control statements?

THREE TYPES OF CONTROL STATEMENTS ARE:

- Selection or branching
- Repetition or looping

- Jump or unconditional branching

2. When do you prefer for loop statement?

When the programmer knows how many times the loop has to be repeated beforehand, he prefers to use for loop instead of while.

3. What is looping?

Repeating a set of statements until a terminating condition is satisfied is known as looping

4. Give the syntax of for statement.

SYNTAX

```
for (initialization;test condition;increment or decrementation)
{
Statement block;
}
```

5. What are Control structures or Control statements?

Control structures are the statements help control the order of execution of statements in a program. They are useful to the programmers to incorporate the following:

- Execution of a statement or a group of statements for a fixed number of times
- Execution of a statement or a group of statements as long as a certain condition is true
- Transfer the control from one point in a program to other
- Termination of execution after a specified point
- Bypass certain statements during the execution of a program

6. How do you categorize the Control structures?

Control structures in C are categorized into two types.

Conditional control structures

- Decision making and branching statement – if, else if, nested-if, switch
- Looping – for, while, do-while

Un-conditional control structures

- Un-conditional jump – goto
- Termination – break, return

- Bypassing – continue

7. Define Conditional control statement.

Those statements, which is used to check the Condition and based on the given condition, carry out specific operations. Such type of statements is known as Conditional Control statement. Conditional Control statement is mainly of three main types. They are

1. Conditional execution
2. Looping
3. Selection

8. Define Unconditional control statement.

The Execution of program can be altered by transferring the control from one place to another in a program without specifying the condition. This statement is known as Unconditional Control statement. GOTO is one of the Unconditional Control statements.

9. Define Decision-making and its types.

The decision-making statement checks the given condition and then executes its sub-block. The decision statement decides the statement to be executed after the success or failure of a given condition. It is also known as conditional control statements.

C provides a variety of conditional control statements such as

1. **if**-statement
2. **if-else** statement
3. **Nested-if** statement
4. **switch** statement

10. Define Branching.

C supports an unconditional control statement, to transfer the control from one point to another in a C program. The goto is a branching statement and requires a label

Syntax: **goto label;**

11. When do you use switch statement?

The switch statement is used when there are multiple statements to be executed and also we do not want to execute statements one after the other.

12. Define Looping and its types.

Looping is a powerful programming technique through which a group of statements is executed repeatedly, until certain specified condition is satisfied. Looping is also

called a repetitive or an iterative control mechanism. The while, do-while and for statements are the three powerful loop control statements available in C.

13. What is the difference between *while* and *do-while* loop statements?

while loop	do-while loop
The condition is tested at the beginning of the loop. Therefore, it is called an entry-controlled loop.	The condition is tested at the end of the loop. Therefore, it is called an exit-controlled loop.
The while loop may not be executed at all	The do-while loop is executed at least once

EX:NO:7	SUM OF SERIES USING FOR AND DO – WHILE LOOP	DATE:
---------	---	-------

AIM :

To write a C program to find the sum of 'n' numbers using for, do – while statements.

ALGORITHM:

Step 1 : Start

Step 2 : Declare the variable n, count, sum=0,op

Step 3 : get the value of n and op

Step 4 : Using switch case select do while loop (or) for loop

Step 5 : **For Loop** : Repeat Step 5 until the condition count<n get false.

- i. Read a number 'n'
- ii. Calculate sum = sum + n

Step 6 : Display Sum value

Step 7 : **do-while Loop** : Repeat Step 7 until the condition count<n get false

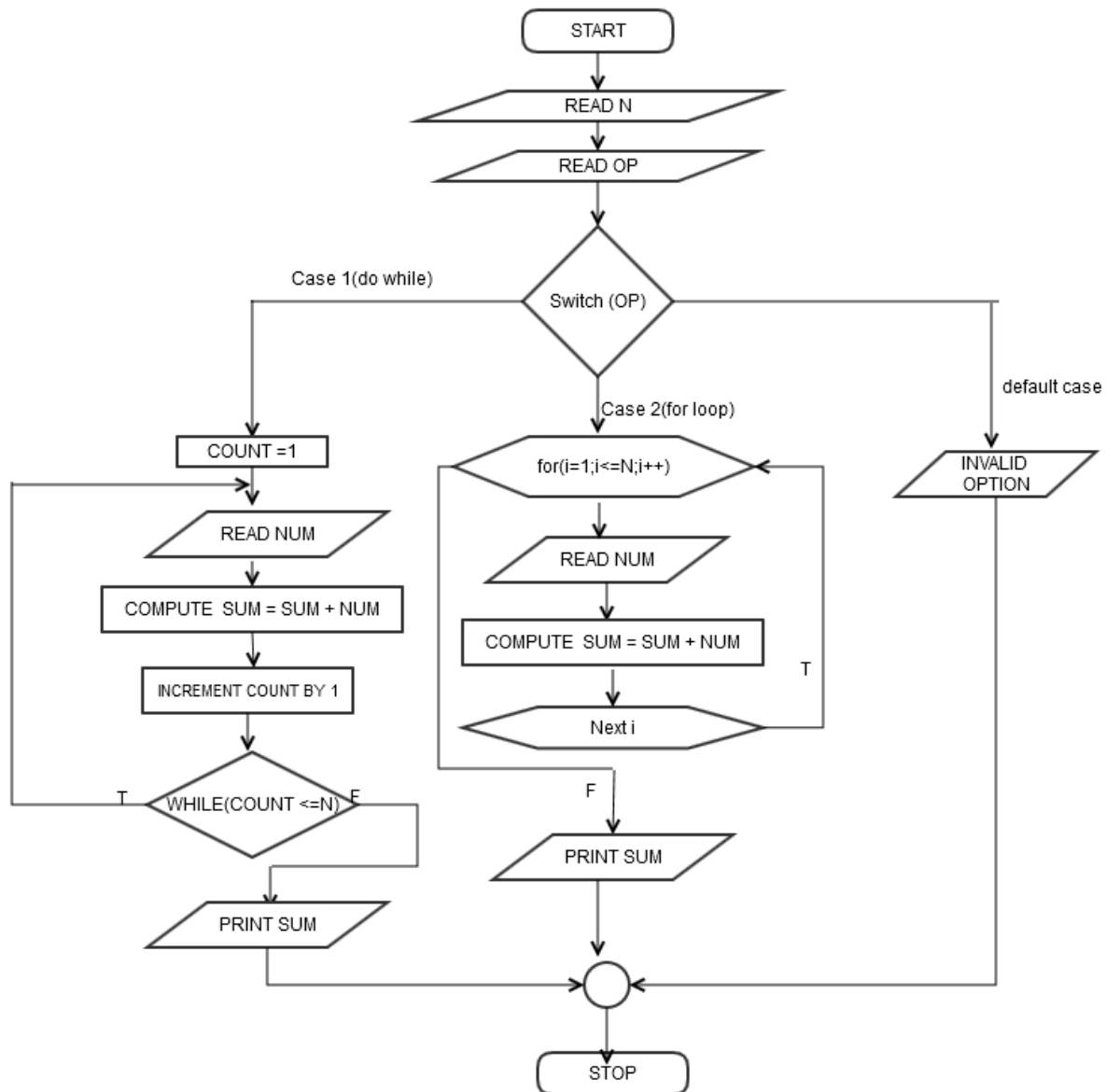
- i. Read a number 'n'
- ii. Calculate sum = sum + n
- iii. Increment Count Value by 1

Step 8 : Display Sum value

Step 7 : In default, display the invalid option message.

Step 8 : Stop execution.

FLOW CHART :



SOURCE CODE :

```
#include <stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    int n, sum = 0, c, value,op;
    printf("\nSUM OF THE SERIES");
    printf("\n*-----*");
    printf("\nEnter the number of integers you want to add :");
    scanf("%d", &n);
    printf("\n Press\n\t 1- do while loop\n\t 2-for loop\nEnter Ur
Choice : ");
    scanf("%d",&op);
    switch(op)
    {
    case 1:
        c=1;
        do
        {
            scanf("%d",&value);
            sum = sum + value;
            c++;
        }while(c <= n);
        printf("Sum of entered integers = %d\n",sum);
        break;
    case 2:
        printf("Enter %d integers\n",n);
        for (c = 1; c <= n; c++)
        {
            scanf("%d",&value);
            sum = sum + value;
        }
        printf("Sum of entered integers = %d\n",sum);
        break;
    default:
        printf("\n\n Invalid Option");
    }
    getch();
}
```

OUTPUT :

SUM OF THE SERIES

Enter the number of integers you want to add :5

Press

1- do while loop

2-for loop

Enter Ur Choice : 1

do-While

Enter 5 Integers

78

66

-34

12

07

Sum of entered integers using do-while loop is 129

RESULT:

Thus the program to find the sum of 'n' numbers using for, do – while statements has been executed and verified successfully.

EX:NO:7SUM OF SERIES USING FOR AND DO – WHILE LOOP

1. State any two uses of decision making statements.

- 1.Improves readability and clarity of the program.
- 2.Reduces testing problems

2. What is looping?

Looping statements are used to execute a group of statements repeatedly until a particular condition is satisfied.

3. Give the syntax of for statement.

SYNTAX

```
for (initialization;test condition;increment or decrementation)
{
    Statement block;
}
```

4. Differentiate between while and do-while loop.

WHILE LOOP	DO WHILE LOOP
1. It checks the condition first and then executes the statement inside the loop 2. It at least executes zero time 3. It does not end with a semicolon (;) 4. It is also called as entry controlled loop 5. <u>SYNTAX</u> while(condition) {	1. It executes the statement first and then checks the condition 2. It at least executes one time 3. It ends with a semicolon (;) 4. It is also called as exit controlled loop 5. <u>SYNTAX</u> do {

Statement block; }	statement block; } while (condition);
-----------------------	---

5. Differentiate between break and continue.

break - The break statement is used to jump out of loop. After the break statement control passes to the immediate statement after the loop.

continue - Using continue we can go to the next iteration in loop.

6. What is entry controlled and exit controlled loop ?

In Entry controlled loop the test condition is checked first and if that condition is true than the block of statement in the loop body will be executed while in exit controlled loop the body of loop will be executed first and at the end the test condition is checked, if condition is satisfied than body of loop will be executed again.

- While – Entry controlled loop
- Do----While loop- Exit controlled loop

7. What are the branching statements equal?

- 1.Switch statement
- 2.else if ladder

8. When is the default statement in the switch statement executed?

When no values of expression match with the case values the statements inside the default are executed

9. What are the two branching statements equal?

- 1.Switch Statement
- 2.else if ladder

10. What is the use of goto statement ?

The goto statement is used to change the order of execution from one part of the program of the program to another part of the program unconditionally.

11. Differentiate forward Jump and Backward Jump.

Forward Jump	Backward Jump
--------------	---------------

Syntax : <pre> goto label; ----- ----- label; statement; </pre>	Syntax: <pre> label; statement; ----- ----- goto label; </pre>
Here, label : is placed after the goto label ; statement. Hence some statements are skipped from execution	Here, label : placed before the goto label ; statement. Hence some statement's are executed repeatedly.

EX:NO:8	FACTORIAL USING FUNCTION	DATE:
----------------	---------------------------------	--------------

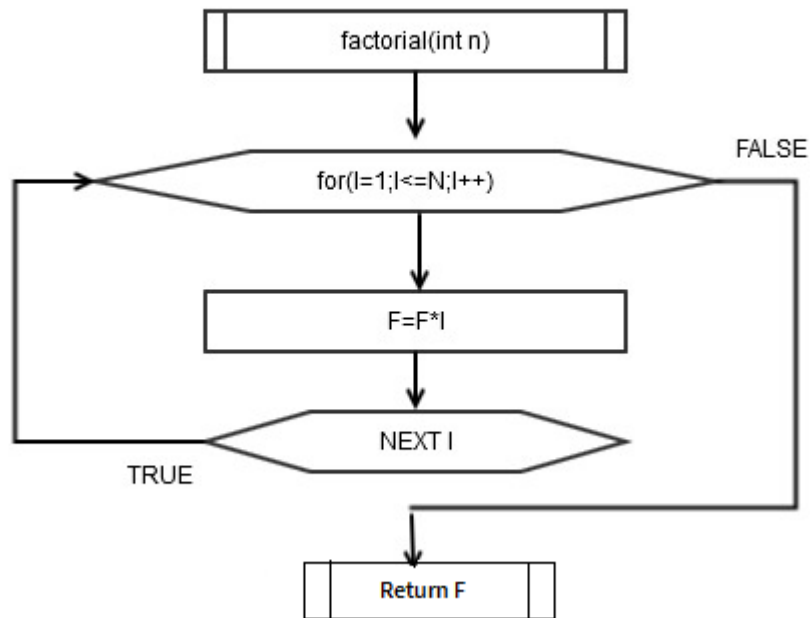
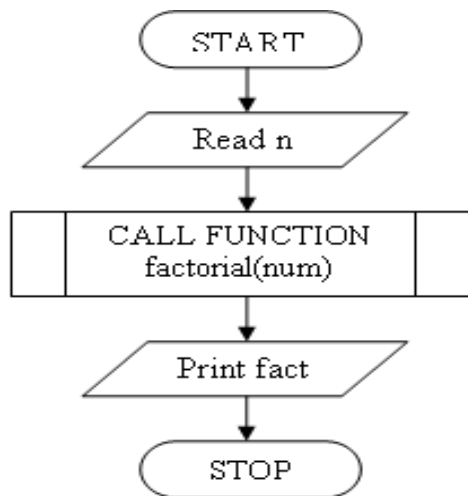
AIM :

To write a C program to find the factorial of a given number using Functions.

ALGORITHM:

- Step 1 : Start
- Step 2 : Declere the function
- Step 3 : Declare the variable i, fact, num
- Step 4 : Get the value of num
- Step 5 : Call the function factorial(num)
- Step 6 : In the called function declare the variable i, f=1
- Step 7 : Using for loop find the factorial using expression $f = f * i$
- Step 8 : Repeat Step7 until for loop get false
- Step 9 : Return to main function
- Step 10 : Stop

FLOW CHART :



SOURCE CODE :

//Program No.8 Factorial Using Function

```
#include <stdio.h>
```

```
#include<conio.h>
```

```
long factorial(int);
```

```
void main()
```

```
{
```

```
    int number;
```

```

long fact = 1;
printf("\nFACTORIAL USING FUNCTION");
printf("\n*-----*");
printf("\nEnter a number to calculate it's factorial : ");
scanf("%d", &number);
printf("%d! = %ld\n", number, factorial(number));
getch();
}

```

```

long factorial(int n)
{
    int c;
    long result = 1;
    for (c = 1; c <= n; c++)
        result = result * c;
    return result;
}

```

OUTPUT:

```

FACTORIAL USING FUNCTION
*-----*
Enter a number to calculate it's factorial : 6
6! = 720

```

RESULT:

Thus the program to find the factorial of a given number using functions has been executed and verified successfully.

EX: NO: 8 FACTORIAL USING FUNCTION

1. What is meant by modularization?

Breaking the bigger problem into smaller sub problems is known as modularization

2. Differentiate between standard functions & user-defined functions?

1. PREDEFINED OR BUILT-IN FUNCTION OR LIBRARY FUNCTION:

The library functions are pre-written and are included in the standard header files

SYNTAX:

```
#include<library function name.h>
```

Eg: #include<math.h>

2. USER DEFINED FUNCTION:

- C language allows the user to define functions
- The complex program is divided into number of small programs
- These small programs are called subprograms or functions

3. Differentiate between actual arguments and formal arguments/parameters.

Actual argument:-

A defined function is called by specifying the name followed by use of arguments or parameters. These arguments are called actual arguments because through these arguments only the formal arguments receives values .

Eg:- add (40,50) ;

FORMAL ARGUMENTS:-

The arguments present in the function definition are called formal arguments (or) dummy arguments because it receives values only from the calling functions

Eg:

```
Void add(int x,int y)
```

```
{  
}
```

4. What is a function?

- A function is a set of instructions to solve a specific and well-defined problem.
- A function is complete and independent (It has its own set of instructions to execute, declaration, beginning and end)
- A function may or may not return a value to the calling program after its execution. If the function to return the value to the calling program, then the return statement has to be used in the function definition.

5. What are the benefits of using functions?

- **Reusability:** A function, once written for solving a specific task, can be used again and again. We need not rewrite it rather reuse it.
- **Build own library:** User can build his/her own library of the most commonly used functions. This reduces the time and space complexity. It also promotes the portability.
- **Debugging is easier:** Since each function is a small module, it is easier to locate the errors and correct them.

6. How function is classified?

Function in C Language is mainly classified into two types. They are as follows,

- Library functions (Built – in functions)
- User-defined functions.

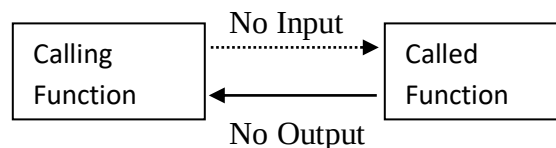
User-defined functions are designed and developed by the user while writing and compiling a C program. (Example: Main function)

7. How do you categorize the function types?

Depending upon the arguments present, return value sends the result back to the calling function. Based on this, the function are divided into four types

- Without arguments and return values
- With arguments but without return values
- Without arguments but with return values.
- With arguments and return values

Eg., for 1:



8. Distinguish between the user-defined functions and the standard functions?

- The **user-defined functions** are those modules which are written by the programmers. They serve some specific purpose. The functions written to find *the volume of a cube*, to find *an area of a circle* etc are the user-defined functions. (Example: Main function)

- The **standard functions** are those functions which are built-in functions. They are developed by the product developers. They are general purpose functions written for specific area of computation. The **sqrt()**, **abs()**, **pow()**, **log()**, etc., are some of the examples of standard functions. The standard functions are called *library functions*.

10. List various library functions available in C language.

Library functions are also known as Built-in functions. The Library functions available in C language are,

- Input/Output functions.
- Boolean functions.
- Conversion functions.
- Memory functions.
- String functions.
- Miscellaneous functions.
- Math or Arithmetic functions and
- Time functions.

11. What are the elements of user-defined functions?

User defined function has three elements. They are

- Function definition (function implementation)
- Function call
- Function declaration

12. What are the elements of function header?

It consists of three parts.

- function type(return type)
- function name
- formal parameter list

13. Explain about function declaration with example.

Like variables, all functions in a C program must be declared, before they are invoked. It was also known as Function prototype. It consists of four parts.

- Function type (return type)
- Function name
- Parameter list
- Terminating semicolon

General form:

datatype function name (argument list);

This is very similar to the function header line except the terminating semicolon.

Examples:

```
double sqroot(int);
void prod(int , int );
int mul(int, int);
```

14. Distinguish between function definition and function declaration?

The function definition and function declaration are different. The function definition means writing of the actual code. But the function declaration is the instruction given to the compiler regarding the functions that appear after the main () function in a C program.

15. What is a calling program and how is different from called program?

- If a given program is divided into the number of small modules called functions, then there may be a communication between some of them.
- A function which calls other function is called a *calling function*. And the one being called is called the *called function*. For example, if a function **funcX()** is called inside the body of function **funcY()**, then the function **funcY()** is a *calling program* and **funcX()** is a *called program*.

EX:NO:9	SWAPPING TWO NUMBERS USING CALL BY VALUE AND CALL BY REFERENCE	DATE:
----------------	---	--------------

AIM :

To write a C program to swap two numbers using call by value and call by reference.

ALGORITHM:

- Step 1 : Start
- Step 2 : Accept A,B
- Step 3 : Declare the function swap1 and swap2
- Step 4 : Print the values of a and b before swapping
- Step 5 : Call swapbyval(A,B)
- Step 6 : Print A and B
- Step 7 : Call swapbyref(&A,&B)
- Step 8 : Print A and B
- Step 9 :Stop

Swapbyval(A,B):

Step 1: Start

Step 2: Interchange the values of A and B by using assignments

$T=A; A=B; B=T$

Step 3 :Stop

Swapbyref(*A,*B):

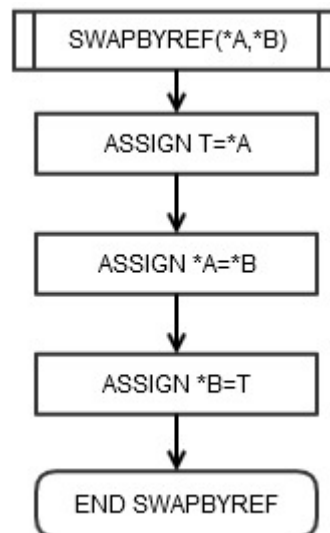
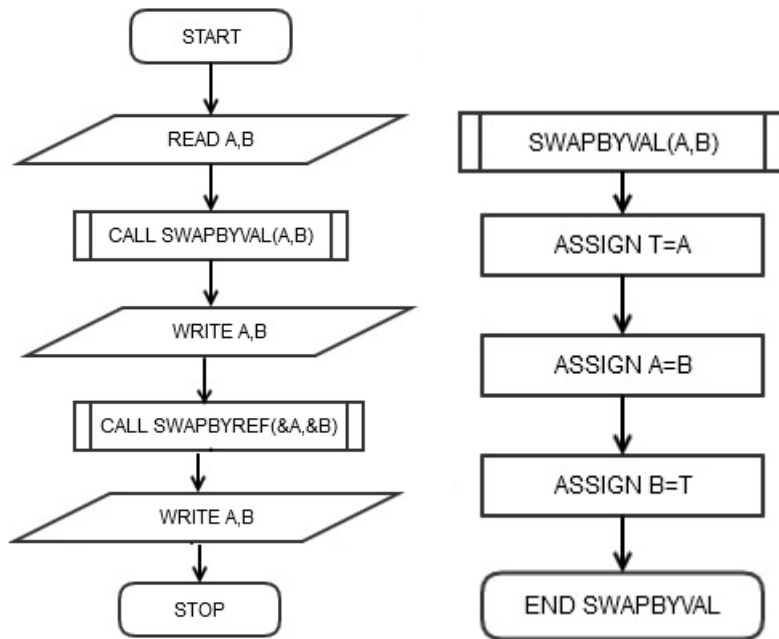
Step 1: Start

Step 2: Interchange the values of A and B by using assignments

$T=*A; *A=*B; *B=T$

Step 3 :Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.9 Swapping using Call by value and call by reference
#include<stdio.h>
#include<conio.h>
void main()
{
    int a, b;
    void swap1(int, int);          // fn declaration for call by value
    void swap2(int *x,int *y);    // fn declaration for call by reference
    printf("\nSwapping Program");
    printf("\n*-*-*-*-*-*-*-*");
    printf("\nEnter A Value:");scanf("%d",&a);
    printf("\nEnter B Value:");scanf("%d",&b);
    swap1(a,b);
    printf("\nIn call by value, After swapping result is:\t a=%d \t b=%d",a,b);
    swap2(&a,&b);
    printf("\nIn call by reference,After swapping result is:\t a=%d \t b=%d",a,b);
    getch();
}
void swap1(int x, int y)          // call by value
{
    int t;
        t=x;
        x=y;
        y=t;
}
void swap2(int *x,int *y)    // call by reference
{
    int t;
    t=*x;
    *x=*y;
    *y=t;
}
```

OUTPUT:

Swapping Program

--*-*-*-*-*-*

Enter A Value:89

Enter B Value:76

In call by value, after swapping result is: a=89 b=76

In call by reference, after swapping result is: a=76 b=89

RESULT:

Thus the program to swap two numbers using call by value and call by reference has been executed and verified successfully.

EX: NO: 9 SWAPPING TWO NUMBERS USING CALL BY VALUE AND CALL BY REFERENCE

1. What is meant by swapping of two variables?

Interchanging the values stored in the two variables

2. How will you call a function ?

A function will be called or invoked by its name. Eg. Add(2,5)

3. What are different ways of calling a function ?

- *Functions with no arguments and no return values*
- *Functions with no arguments and with return values*
- *Functions with arguments and no return values*
- *Functions with arguments and return values*

4. What are the 3 factors to be considered while using function ?

Prototype declaration of function, function definition and function call

5. What is a pointer?

A pointer is a variable which holds the address of other variable. Apart from the elements of basic data type pointers point to the different data items such as arrays, structures and functions that are used in the program.

The operators used with the pointers are the address operator (&) and the dereferencing operator (*). The address operator returns the memory location of a particular element and the dereferencing operator (indirection operator) specifies the value housed at that memory location.

6. Differentiate between address operator and dereferencing operator.

Returns address of the given variable. Consider the following example, int *ptr, var = 10;	The dereference operator is an unary operator which returns the value of the pointee by dereferencing the value(address) in pointer
---	---

```
ptr = &var;
```

ptr = &var sets the address of the variable var to pointer ptr. & is also called as reference operator.

Let us try to understand the purpose of reference operator using the following example program.

```
#include <stdio.h>

int main() {
    int var = 10, *ptr;

    /* assigning address of variable var to
pointer ptr */
    ptr = &var;
    /* printing the address of the variable var */
    printf("Address of var is 0x%x\n", &var);
    /* printing the value of pointer ptr */
    printf("Value of ptr is 0x%x\n", ptr);
    return 0;
}
```

variable. Consider the following example,

```
int var1 = 10, var2;
int *ptr; // '*' used for declaration
ptr = &var;
var2 = * ptr; // '*' used for deferencing
```

The above statement(var2 = *ptr) dereferences the address in ptr and assigns the pointee value(value of var1) to var2. Now, the value of var2 becomes 10. * is also called as dereference operator.

Let us try to understand the purpose of dereference operator using the following example program.

```
#include <stdio.h>

int main() {
    int var1 = 10, var2;
    int *ptr;

    /* assigning the address of variable var1 to
pointer ptr */
    ptr = &var1;
    /* assigning dereferenced value to var2 */
    var2 = *ptr;

    /* printing addresses and values of the
variables */
    printf("Address of var1 is 0x%x\n",
```

	<pre> &var1); printf("Address of var2 is 0x%x\n", &var2); printf("Value of ptr is 0x%x\n", ptr); printf("Value of var1 is %d\n", var1); printf("Value of *ptr is %d\n", *ptr); printf("Value of var2 is %d\n", var2); return 0; } </pre>
--	--

7. How do you declare a pointer variable?

Int * a,*b;

8. What is the difference between call by value and call by reference ?

Call By Value:-

In method ,when a function is called only one copy of the values of actual arguments are passed to the called functions . That is the actual and formal arguments use different memory locations. Hence any changes in the formal arguments do not affect the value of actual arguments .

Call By Reference:-

In this method, when a function is called, the address of actual arguments is passed to the called function. So the called functions refer the original values of the actual arguments.Hence any changes in the formal arguments affect the value of actual arguments.

9. What is an address operator ?

An address-of operator is a mechanism within C++ that returns the memory address of a variable. These addresses returned by the address-of operator are known as pointers, because they "point" to the variable in memory.

The address-of operator is a unary operator represented by an ampersand (&). It is also known as an address operator.

10. What is int *a, float *b, char *m ?

Int *a – pointer to interger variable

Float *b – pointer to float variable

Char *m – pointer to character variable

EX:NO:10	SMALLEST AND LARGEST ELEMENT IN AN ARRAY	DATE:
----------	--	-------

AIM :

To write a C program to find the smallest and largest element in an array.

ALGORITHM:

Step 1 : Start

Step 2 : Declare array a[50]

Step 3 : Read Elements for array a[] with size n

Step 4: Assign big = a[0]

Step 5 : Using for loop,compare if (big < a[i]) ,then assign big = a[i]
otherwise iterate

the loop

Step 6: Assign small = a[0]

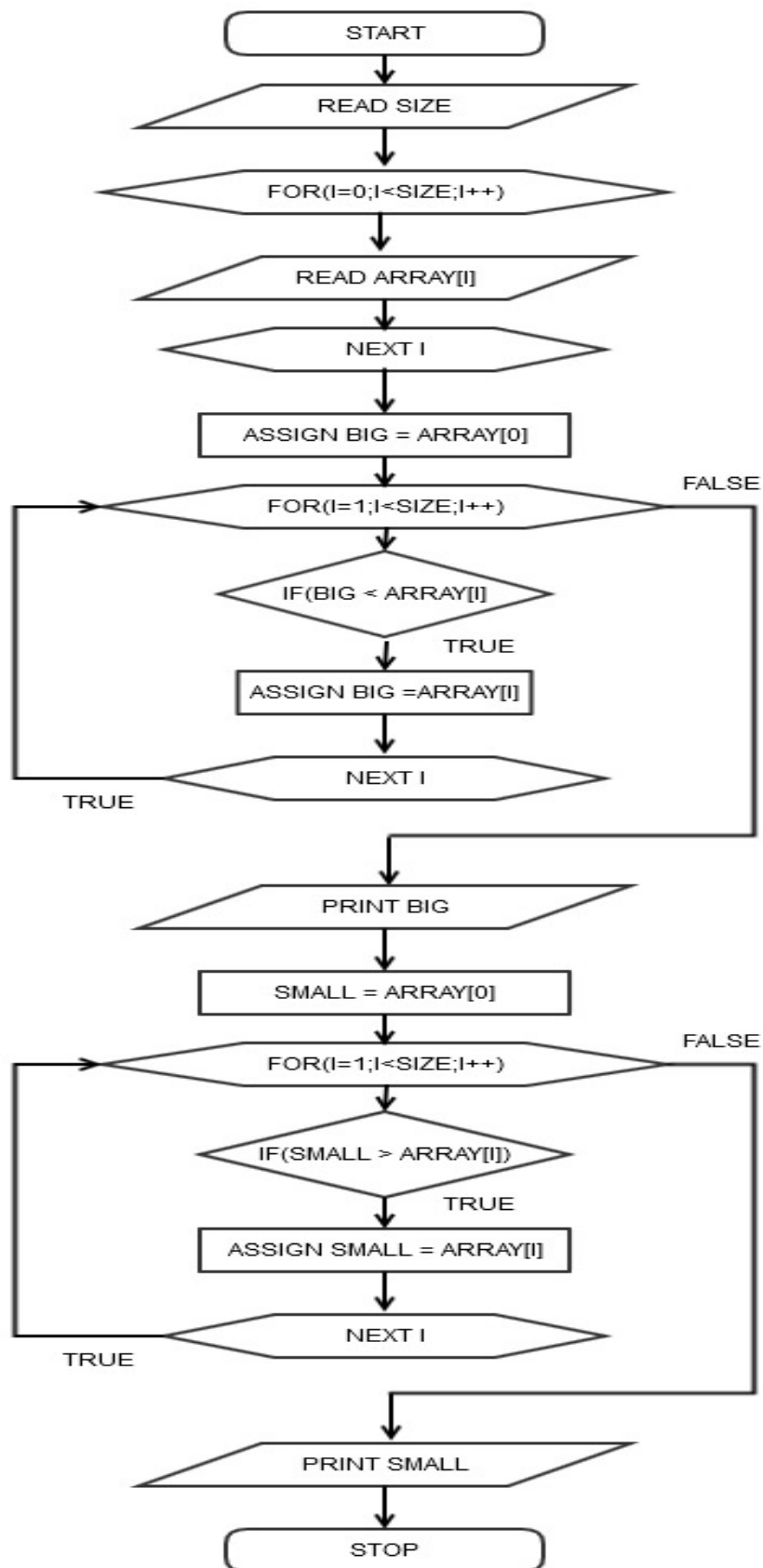
Step 7 : Using for loop,compare if (small > a[i]) ,then assign small =
a[i] otherwise

iterate the loop

Step 8 : Print the big and small value.

Step 9 : Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.10 Largest and Smallest
#include<stdio.h>
#include<conio.h>
void main(){
    clrscr();
    int a[50],size,i,big,small;
    printf("\nSmallest and Largest Element in an array");
    printf("\n*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*");
    printf("\nEnter the size of the array: ");
    scanf("%d",&size);
    printf("\nEnter %d elements in to the array: ", size);
    for(i=0;i<size;i++)
        scanf("\n%d",&a[i]);
    big=a[0];
    for(i=1;i<size;i++)
    {
        if(big<a[i])
            big=a[i];
    }
    printf("\nLargest element : %d",big);
    small=a[0];
    for(i=1;i<size;i++)
    {
        if(small>a[i])
            small=a[i];
    }
    printf("\nSmallest element: %d",small);
    getch();
}
```

OUTPUT:-

```
Smallest and Largest Element in an array
*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
Enter the size of the array: 5
Enter 5 elements in to the array: 78
65
-100
32
99
```

Largest element : 99
Smallest element: -100

RESULT:

Thus the program to find the smallest and largest element in an array has been executed and verified successfully.

EX:NO:10 SMALLEST AND LARGEST ELEMENT IN AN ARRAY

1. What is an array?

Array is a collection of same data items that share a common name. An array can be created with any data type

2. Give the classification of arrays?

One dimensional, two dimensional, multi dimensional

3. Differentiate between an array and an ordinary variable.

Array variable is denoted with subscript. Ordinary variable is denoted without subscript. Array variable is also called as subscripted variable.

4. How will you declare an integer array of size 10?

```
Int a[10];
```

5. What is an index?

Subscript is also known as index

6. Which variable is known as subscripted variable ? Give eg.

Array element is known as subscripted variable. Eg a[i]

7. What is static array and dynamic array?

If the number of elements in the array fixed at compile time is known as static array. If the number of elements in the array can be changed at run time is known as dynamic array

8. What is pointer to an array?

The variable which holds the address of array is known as pointer to an array. Eg.

```
Int *a;
```

```
Int b[10];
```

```
a=b;
```

9. What is an array of pointers?

It is an array used for storing a list of addresses. Eg `int *a[10]`

Addresses of 10 integer variables are stored in array called 'a'

10. Define the term Subscript

Subscript is an index number which is used to access the array elements.

11. Write down the Syntax for two dimensional array declaration.

`datatype arrayname[rowsize][columnsize];`

12. How many memory locations are allowed for the following array `a[5][3]`?

15 Memory Locations

EX:NO:11	MATRIX MULTIPLICATION	DATE:
-----------------	------------------------------	--------------

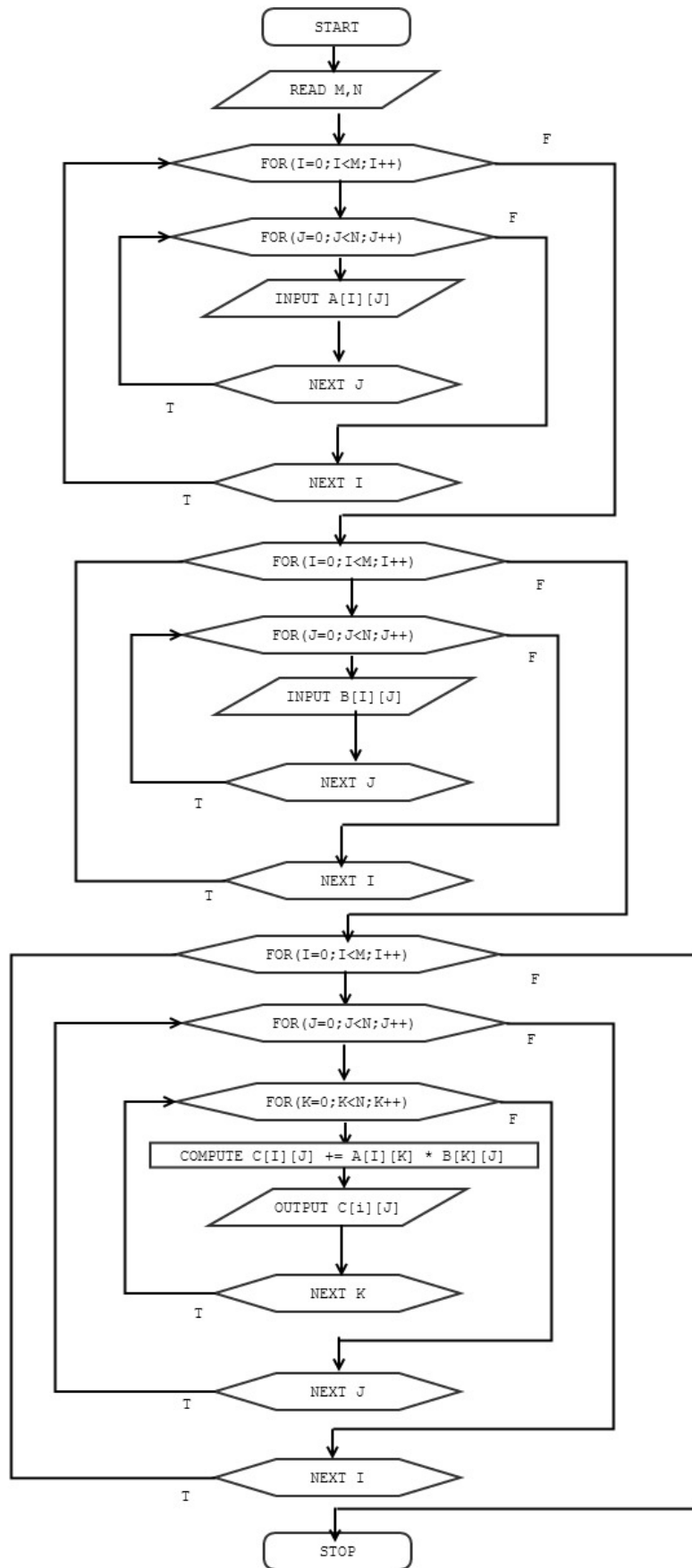
AIM :

To write a C program to perform matrix multiplication.

ALGORITHM:

- Step 1. Start
- Step 2. Define the order of matrix as 3x3
- Step 3. Read the value of two matrices A,B
- Step 4. Set loop `i=0 to 3` `j=0 to 3` and `k=0 to 3`
- Step 5. Calculate `c[i][j]=c[i][j]+a[i][k]*b[k][j]`
- Step 6. End loop
- Step 7. Print `c[i][j]`
- Step 8. Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.11 Matrix Multiplication
#include <stdio.h>
#include<conio.h>
void main()
{
clrscr();
int i, j, m, n, k, a[5][5], b[5][5], c[5][5] = {0};
printf("\n\t\t\tMATRIX MULTIPLICATION ");
printf("\n\t\t\t*-*-*-*-*-*-*-*-* ");
printf("\nEnter order of matrix (m X n): ");
scanf("%d%d",&m,&n);
printf("\n");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
printf("\nEnter elements of Matrix A [%d][%d]: ",i+1,j+1);
scanf("%d", &a[i][j]);
}
}
printf("\n");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
printf("\nEnter elements of Matrix B [%d][%d]: ",i+1,j+1);
scanf("\t%d", &b[i][j]);
}
}
printf("\n\t The resultant Matrix is");
printf("\n\t ----- \n\t");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
for (k = 0; k < n; k++)
c[i][j] += a[i][k] *b[k][j];
printf("\t%d", c[i][j]);
}
```

```

}
printf("\n\t");
}
getch();
}

```

OUTPUT:

```

                                MATRIX MULTIPLICATION
                                *_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*
Enter order of matrix (m X n): 2 2

Enter elements of Matrix A [1][1]: 2
Enter elements of Matrix A [1][2]: 2
Enter elements of Matrix A [2][1]: 2
Enter elements of Matrix A [2][2]: 2

Enter elements of Matrix B [1][1]: 3
Enter elements of Matrix B [1][2]: 3
Enter elements of Matrix B [2][1]: 3
Enter elements of Matrix B [2][2]: 3

```

The resultant Matrix is

```

      12      12
      12      12

```

RESULT:

Thus the program to perform matrix multiplication has been executed and verified successfully.

EX:NO:11 MATRIX MULTIPLICATION

1. What is an array?

Array is a collection of same data items that share a common name. An array can be created with any data type

2. Give the classification of arrays?

One dimensional, two dimensional, multi dimensional

3. Differentiate between an array and an ordinary variable.

Array variable is denoted with subscript. Ordinary variable is denoted without subscript. Array variable is also called as subscripted variable.

4. What is two dimensional array?

Two Dimensional Array: Two dimensional array is an array in which each element referred by two subscripts. It is like a table containing rows and columns. It is known as matrix.

Syntax:

```
datatype arrayname[row_size][column_size];
```

where

datatype-built in datatype(int,float,double,etc)

arrayname-it is identifier

rowsize-number of rows

columnsize-number of columns

5. How will you declare a two dimensional array?

Example:

```
int a[2][3];
```

6. What is row major method of storing array elements?

Storing the elements in row order. After storing I row elements moving to the II row then III row etc.

7. What is the use of printf(“\n”) statement ?

\n – new line character. After printing the current line, print header moves to the next line

8. What is nested loop?

Loop within a loop

9. What is a running variable? Can you give same name for running variables in a nested loop?

Variable used as a counter in for loop is known as running variable. We can not give the same name to both the running variables in an nested loop

10. Write the statements for printing the elements of a nxn matrix.

```
for(i = 0; i<m;i++)
{
    for(j=0;j<n;j++)
        { printf("%d",a[i][j]);
          }
    printf("\n");}
```

EX:NO:12	USAGE OF LOCAL AND GLOBAL VARIABLES	Date:
-----------------	--	--------------

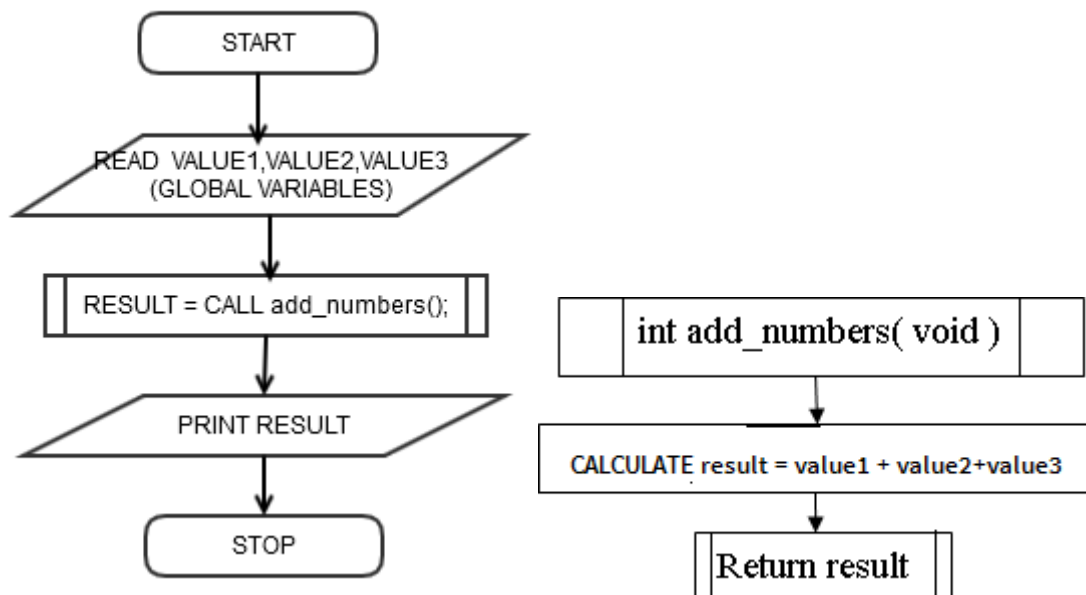
AIM :

To write a C program to demonstrate the usage of Local and Global variables.

ALGORITHM:

- Step 1 : Start
- Step 2 : Declare the function add_numbers and global variables.
- Step 3 : In function definition, calculate result=value1+value2+value3 and return it.
- Step 4 : With in main, declare the local variable result.
- Step 5 : Assign value1=10,value2=20,value3=30
- Step 6 : Add the values and display the result
- Step 7 : Stop

FLOW CHART :



SOURCE CODE :

```

//Program No.12 Local and Global Variable
#include <stdio.h>
#include<conio.h>
int add_numbers( void );
int value1, value2, value3;           // Global variable
int add_numbers( void )
{
    auto int result;                 // Local variable
    result = value1 + value2 + value3;
    return result;
}
void main()
{
    clrscr();
    auto int result;                 // Local variable
    printf("\nProgram for Local and Global Variable");
  
```

```

printf("\n*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*");
printf("\nEnter Value1:");scanf("%d",&value1);
printf("\nEnter Value2:");scanf("%d",&value2);
printf("\nEnter Value3:");scanf("%d",&value3);
result = add_numbers();
printf("\nThe sum of %d , %d and %d using Local and Global variable
is : %d ",value1, value2, value3, result);
gtech();
}

```

OUTPUT:

```

Program for Local and Global Variable
*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*

```

```

Enter Value1:56

```

```

Enter Value2:77

```

```

Enter Value3:12

```

```

The sum of 56 , 77 and 12 using Local and Global variable is : 145

```

RESULT:

Thus the program to demonstrate the usage of Local and Global variables has been executed and verified successfully.

EX:NO:12 USAGE OF LOCAL AND GLOBAL VARIABLES

1. What is a variable ?

- a. A variable is a value that can change any time. It is a memory location used to store a data value.
- b. Variables can be of different data types. The data types are integers, real or character constants
- c. A variable is a data name used for storing a data values.

Rules for defining variables:

- a. White space is not allowed
- b. The length of a variable must not be more than 8 characters
- c. A variable should not be a Keyword
- d. The variable names may be a combination of upper and lower characters.
- e. The variable name should not start with a digit

Example of variable names are

- 1.Sun
- 2.number
- 3.Salary
- 4.Emp_name
- 5.average1

Examples of Invalid Variable names are

- 1.123
- 2.(area)
- 3.6th
- 4.%abc

2. Differentiate between local and global variables.

Local variables	Global variables
Local variables are the data declared within a user defined function and available only within that function.	Global variables are the data that are declared before the main program and that are available to the main program as well as the functions that are called from the main program.
Ex: int sum(int a, int b) { int r; 	 int p,r; void main() { }

}	
---	--

3. What do you mean by storage classes?

By default, for every variable declared, some storage classes are assumed by the compiler. To define any variable, we need to mention its data type and its storage class.

4. What are the different types of storage classes?

There are 4 data types of storage classes in C language:

1. Auto (automatic: default storage class)
2. Static
3. Register
4. Extern

5. What do you mean by parameter passing?

- Parameter passing is a mechanism through which the values of the arguments are passed onto a function.
- The called function receives these arguments in the form of parameters (variables that hold the values of the arguments) and uses them for a specific computation.

6. List the different types of parameter passing mechanisms in C?

These are two types of parameter passing mechanisms in C.

- Call-by-value

- Call-by-reference

7. Distinguish between call-by-value and call-by-reference parameter passing mechanisms?

Call-by-value	Call-by-reference
In this method , the values of the arguments are passed from a calling function to a called function	In this method, the addresses of the arguments are passed from a calling function to a called function.
The values are copied into the called function	No values are copied into the called function
Any changes made to these values in the called function, will not impact the original values in the calling function	Any changes made to these values in the called function, will modify the original values in the calling function

8. What does the storage class tell the compiler?

The storage class of a variable tells the compiler

- Storage area of the variable
- Initial value of the variable if not initialized
- Scope of the variable
- Life of the variable that is how long the variable would be active in the program.

9. List the four storage class specifiers in C.

The four storage class specifiers of C are:

- auto (automatic variables)
- static (static variables)
- extern (extern variables)
- register (register variables)

10. What is recursion?

Recursion is the name given to the technique of defining a function in terms of itself. A recursive function calls and recalls itself, till it reaches a stopping condition.

EX:NO:13	STRING HANDLING FUNCTIONS	DATE:
-----------------	----------------------------------	--------------

AIM :

To write a C program to perform various string handling functions: strlen, strcpy, strcat, strcmp.

ALGORITHM:

Step 1 : Start

Step 2 : Accept two strings s1,s2

Step 3 : Calculate the length of string s1 using strlen(s1)

Step 4 : Copy the string (s1) in new string(dest) using strcpy(dest,s1)

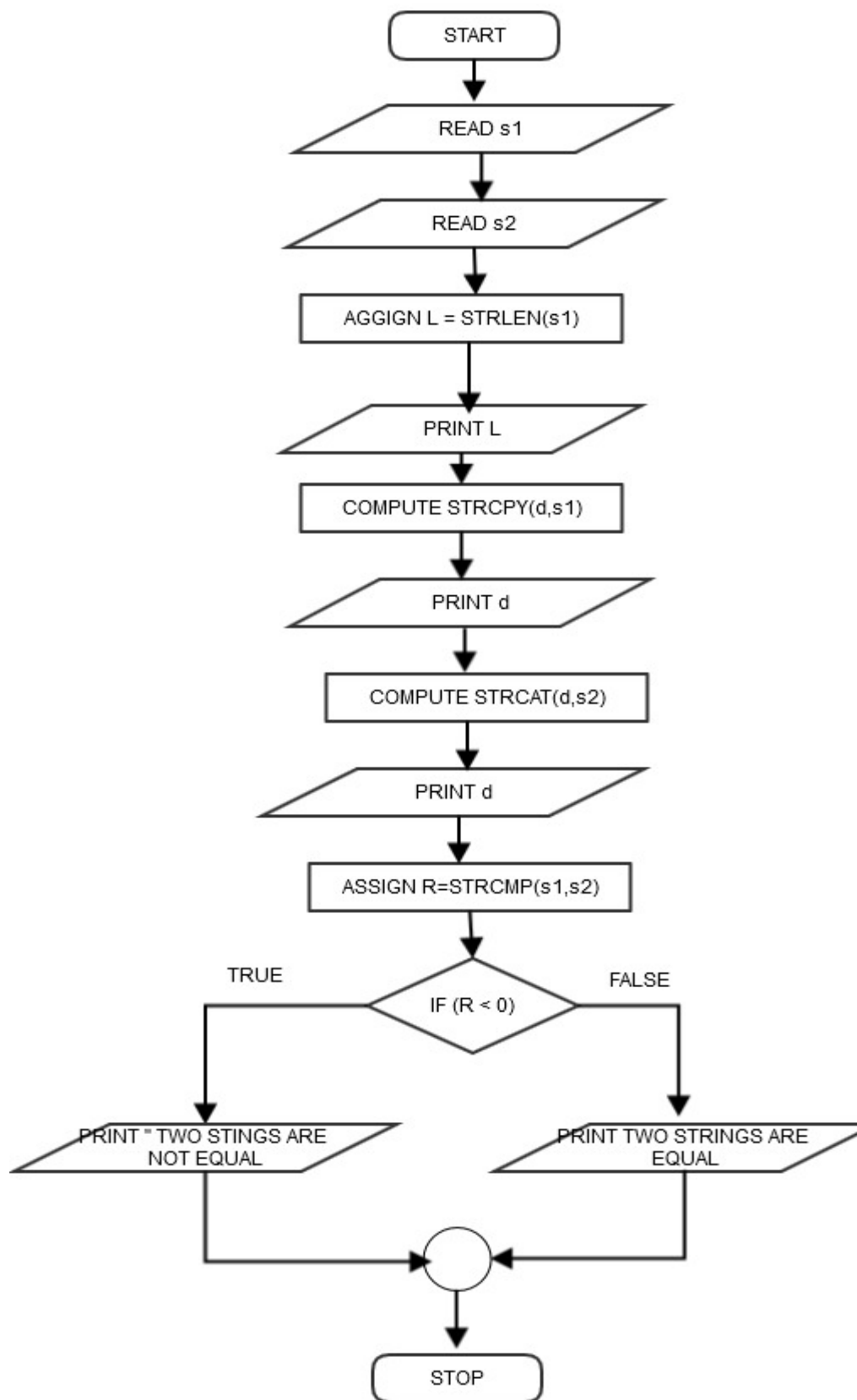
Step 5 : Combine two string using strcat(dest,s2)

Step 6 : Compare two string using strcmp(s1,s2) and return signed integer values .If

the result value is negative then two strings are unequal otherwise equal.

Step 7 : Stop

FLOW CHART :



SOURCE CODE :

```
#include<stdio.h>
#include<string.h>
#include,conio.h>
void main()
{
    clrscr();
    char s1 [25],s2[25],d[25];
    int l,r;
    //Input here
    printf("\nSTRING MANIPULATION");
    printf("\n*-*-*-*-*-*-*-*");
    printf("\nEnter First string:");
    scanf("%s",s1);
    printf("\nEnter Second string:");
    scanf("%s",s2);
    //-----
    //1.String Length
    l = strlen(s1);
    printf("\n1.Length of %s is : %d",s1,l);
    //-----
    //2.String Copy
    strcpy(d,s2);
    printf("\n2.String %s is copied in variable d is :%s",s2,d);
    //-----
    //3.String Concatenation
    strcat(d,s1);
    printf("\n3.After Concatenation : %s",d);
    //-----
    //4.String Comparison
    r=strcmp(s1,s2);
    if(r<0)
        printf("\n4.The strings %s,%s are not equal.",s1,s2);
    else
```

```
printf("\n4.The strings %s,%s are equal.",s1,s2);
getch();
}
```

OUTPUT:

STRING MANIPULATION

*_*_*_*_*_*_*_*_*_*_*_*

Enter First string:INDIA

Enter Second string:PONDY

- 1.Length of INDIA is : 5
- 2.String PONDY is copied in variable d is :PONDY
- 3.After Concatenation : PONDYINDIA
- 4.The strings INDIA,PONDY are not equal.

RESULT:

Thus the program to perform various string handling functions: strlen, strcpy, strcat, strcmp has been executed and verified successfully.

EX:NO:13 STRING HANDLING FUNCTIONS

1. Differentiate between standard functions & user-defined functions?

1.PREDEFINED OR BUILT- IN FUNCTION OR LIBRARY FUNCTION:

The library functions are pre-written and are included in the standard header files

SYNTAX:

```
#include<library function name.h>
```

Eg: #include<math.h>

2.USER DEFINED FUNCTION:

- C language allows the user to define functions
- The complex program is divided into number of small programs
- These small programs are called subprograms or functions

2. Name the different string handling functions?

Function	Work of Function
strlen()	Calculates the length of string
strcpy()	Copies a string to another string
strcat()	Concatenates(joins) two strings
strcmp()	Compares two string
strlwr()	Converts string to lowercase
strupr()	Converts string to uppercase

3. Give the syntax of strlen()

1 strlen() - calculates the length of string. It is defined under "string.h" header file. It takes only one argument, i.e, string name. This function returns only the integer value.

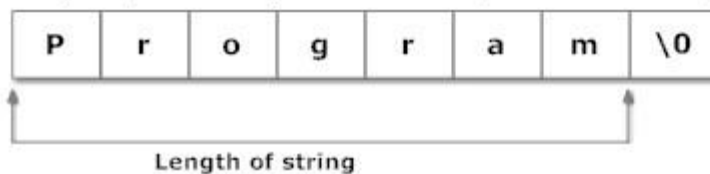
Syntax :

temp_variable = strlen(string_name);

```
char c[]={'P', 'r', 'o', 'g', 'r', 'a', 'm', '\0'};
```

```
temp= strlen(c);
```

Then, temp will be equal to 7 because, null character '\0' is not counted.



4. How will you convert upper case characters in the given string into lower case ?

strupr() -In C programming, strupr() function converts all the lowercase characters in that string to uppercase characters. The resultant from strupr() is stored in the same string.

Syntax : *strupr(string_name);*

Example of strupr()

<pre>#include <stdio.h> #include <string.h> int main(){ char str1[]="UppEr Case"; puts(strupr(str1)); //Converts to uppercase and displays it. return 0; }</pre>	Output UPPER CASE
--	-----------------------------

5. How will you convert lower case characters in the given string into upper case ?

strlwr() - strlwr() function converts all the uppercase characters in that string to lowercase characters. The resultant from strlwr() is stored in the same string.

Syntax :strlwr(string_name);

6. What is the use of #include string.h?

A string in the C language is simply an array of characters. Strings must have a NULL or \0 character after the last character to show where the string ends. A string can be declared as a character array or with a string pointer. First we take a look at a character array example:

```
char mystr[20];
```

7. What is the syntax of strcpy()?

strcpy()- copies the content of one string to the content of another string. It is defined under "string.h" header file. It takes two arguments.

Syntax :strcpy(destination,source);

Here, source and destination are both the name of the string. This statement, copies the content of string source to the content of string destination.

8. Explain strcmp() with eg.

Syntax: temp_varaible=strcmp(string1,string2);

Example of strcmp()

<pre>#include <stdio.h> #include <string.h> int main(){ char str1[30],str2[30]; printf("Enter first string: "); gets(str1); printf("Enter second string: "); gets(str2); if(strcmp(str1,str2)==0) printf("Both strings are equal"); else printf("Strings are unequal"); return 0; }</pre>	Output Enter first string: Apple Enter second string: Apple Both strings are equal.
---	--

Note :

char str1[]="and",str2[]="cat";if(strcmp(str1,str2)) return -1

char str1[]="cat",str2[]="and";if(strcmp(str1,str2)) return 1

char str1[]="and",str2[]="and";if(strcmp(str1,str2)) return 0

9. Give the usage of strrev()

This function is used for reversing the given string

10. Expand ASCII

American Standard Code for Information Interchange

11. How will you get the ASCII value of a given character ?

Asc('A') returns 65

12. What is the use of atoi()?

This function converts the ascii value into integer for usage in arithmetic calculation

EX:NO:14	REMOVAL OF SPECIAL CHARACTERS	DATE:
----------	-------------------------------	-------

AIM :

To write a C program to remove all characters in a string except alphabets.

ALGORITHM:

Step 1 : Start

Step 2 : Read a String str.

Step 3 : Assign i=j=0

Step 4 : Using loop check whether (str[i] != '\0'). If it is true then do the following

- ✓ For Uppercase alphabets, check if ((str[i] >=65) && (str[i] <= 90))
 - If it is true then assign dest[j] = str[i].
 - Increment j value by 1
- ✓ End IF Condition
- ✓ For Lowercase alphabets, check if ((str[i] >=97) && (str[i] <= 122))
 - If it is true then assign dest[j] = str[i].
 - Increment j value by 1
- ✓ End IF Condition
- ✓ Increment i value by 1.

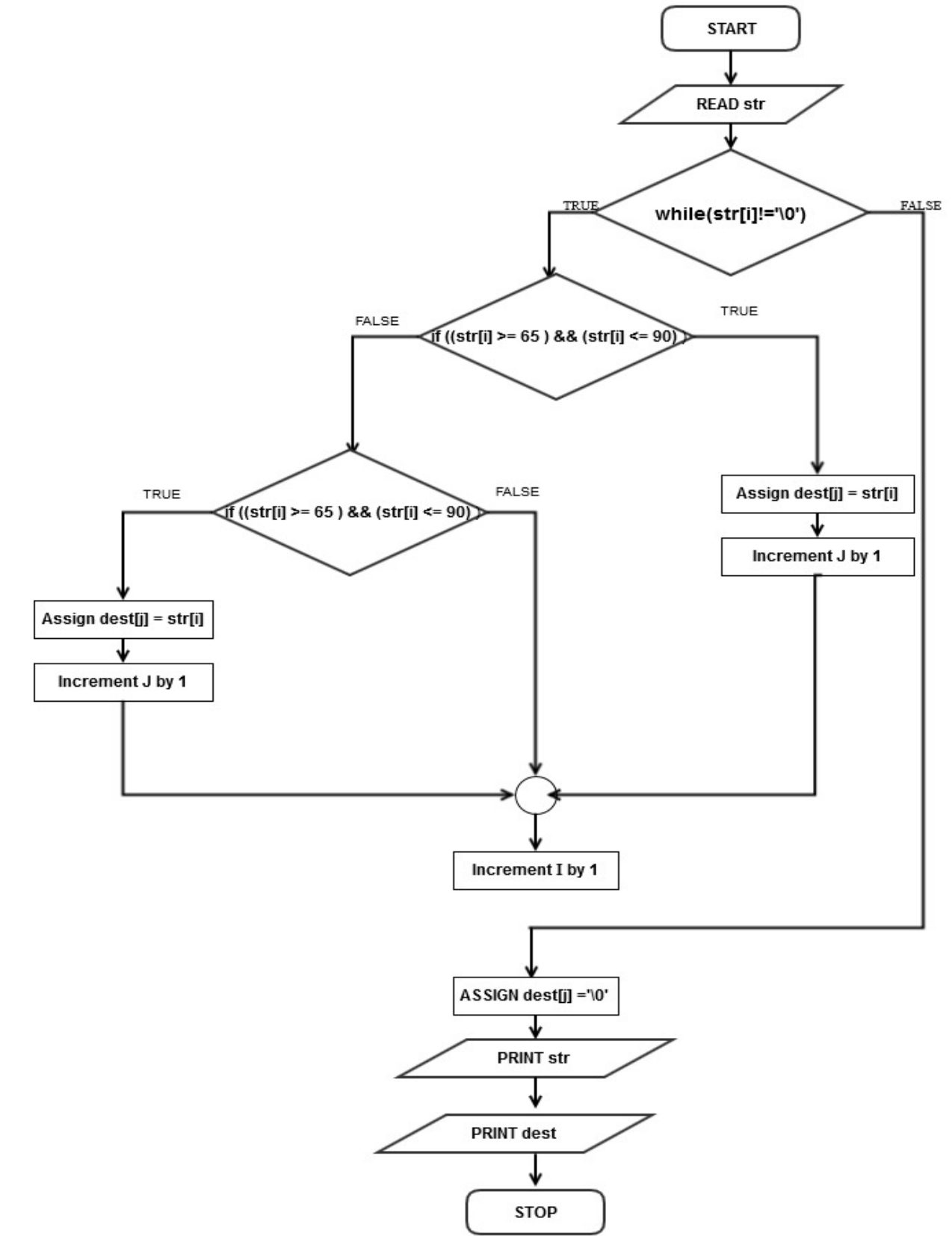
Step 5 : For Termination of new string dest, assign '\0' at the last. (dest[j] = '\0')

Step 6 : Display the original String str

Step 7 : Display Special Characters Removed String dest

Step 8 : Stop

FLOW CHART :



SOURCE CODE :

//Program No.14 Removal of Special Characters from a given string

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    char str[50];
    char dest[50];
    int i,j;
    printf("\nEnter the String:");
    gets(str);
    i=0;
    j=0;
    while(str[i]!='\0')
    {
        if ((str[i] >= 65 ) && (str[i] <= 90) )
        {
            dest[j] = str[i];
            j++;
        }
        if ((str[i] >= 97 ) && (str[i] <= 122) )
        {
            dest[j] = str[i];
            j++;
        }
        i++;
    }
    dest[j]='\0';
    printf("\nOriginal Text :%s",str);
    printf("\nOutput Text:%s",dest);
    getch();
}
```

OUTPUT:-

SPECIAL CHARACTERS REMOVAL IN A STRING

--*-*-*-*-*-*-*-*-*-*-*-*-*-*

Enter the String:E-Mail:rgcetpdy@sify.com

Original Text :E-Mail:rgcetpdy@sify.com

Output Text:EMailrgcetpdysifycom

RESULT:

Thus the program to remove all characters in a string except alphabets has been executed and verified successfully.

EX:NO:14 REMOVAL OF SPECIAL CHARACTERS

1. What is recursion?

- Recursion is the name given to the technique of defining a function in terms of itself.
- A recursive function calls and recalls itself, till it reaches a stopping condition.

2. Define string.

Names are purely a sequence of alphabets. This sequence of alphabets (that are printable ASCII characters) is called **string**. A string is a one-dimensional array of characters. For example, the word “**computer**” is a string of 8 characters. Every string is terminated with the ‘\0’ (NULL) character.

3. What is a character array?

Like numeric arrays, the character arrays are classified into one-dimensional arrays (string) and two-dimensional arrays. Each element of a one-dimensional character array is an ASCII printable character. But, each element of a two-dimensional array is a string, which in turn is a one-dimensional array of characters.

4. When do you use two-dimensional character array?

Besides data base applications, another common application of two dimensional arrays is to store an array of strings. In this section we see how an array of strings can be declared and operations such as reading, printing and sorting can be performed on them.

A string is an array of characters; so, an array of strings is an array of arrays of characters. Of course, the maximum size is the same for all the strings stored in a two dimensional array. We can declare a two dimensional character array of MAX strings of size SIZE as follows:

```
char names[MAX][SIZE];
```

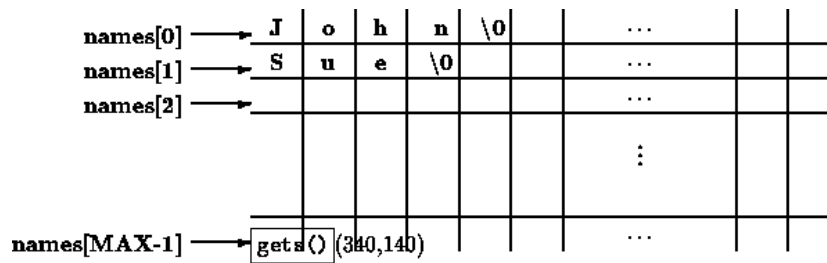


Figure 9.13: An Array of Strings

5. What is the difference between char and string data type?

The char type store only one symbol of data (letter, digit, space, tab and so on). The string type is able to keep virtually data of any length. Also string type is based on arrays and uses reference type working with memory.

6. How will you declare char data type ? Give eg.

Keyword char is used for declaring the variable of character type. For example:

```
char var4='h';
```

Here, *var4* is a variable of type character which is storing a character 'h'.

The size of char is 1 byte. The character data type consists of ASCII characters.

Each character is given a specific value. For example:

For, 'a', value =97

For, 'b', value=98

For, 'A', value=65

For, '&', value=33

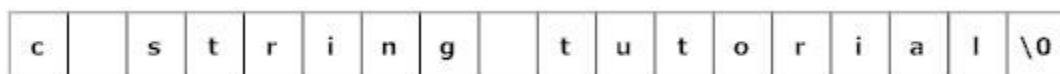
For, '2', value=49

7. How will you declare string data type ? Give eg.

In C programming, array of character are called strings. A string is terminated by null

character /0. For example: *"c string tutorial"*

Here, *"c string tutorial"* is a string. When, compiler encounters strings, it appends null character at the end of string.



8. What is the difference between getchar() and putchar()?

getchar() -- Returns the next character from the standard input (stdin).

putchar(int character) -- Writes character to the current position in the standard output (stdout) and advances the internal file position indicator to the next position.

9. What is the difference between gets() and puts()?

gets(): This function is used for accepting any string through stdin(keyboard) until enter key is pressed. The header file stdio.h is needed for implementing the above function.

<pre>#include<stdio.h> Void main() { Char ch[30]; Printf("Enter the string:"); gets(ch); printf("\n Entered string:%s",ch);}</pre>	Output: Enter the string:Charles Entered string: Charles
--	---

puts(): This function prints the string or character array.

<pre>#include<stdio.h> void main() { char ch[30]; printf("Enter the string:"); gets(ch); puts("Entered String:"); puts(ch);}</pre>	Output: Enter the string:Tommy Dreamer Entered string: Tommy Dreamer
--	---

The predefined functions `gets()` and `puts` in C language to read and display string respectively.

<pre>int main(){ char name[30]; printf("Enter name: "); gets(name); //Function to read string from user. printf("Name: ");</pre>	<pre>puts(name); //Function to display string. return 0; }</pre> Output Enter name: Tom Hanks Name: Tom Hanks
--	--

10. What are the functions available for getting a single char from the user?

- `getchar()`
- `getch()`
- `getche()`

- d. `getc()`
- e. `scanf("%c",&ch);`

11. What is the difference between `getch()` and `getche()`?

`getch()` : Reads a character and never waits for Enter key. Just gets processed after getting any key pressed and it never echoes the character on screen which is pressed.

`getche()` : it works same as `getch()` but it echoes on screen.

12. Write the coding for copying the given string character by character into an array.

```
size_t len = strlen (str) + 1;
char * ncstr = malloc (len);
strcpy (ncstr, str);
```

13. What is meant by ASCII value of a character?

Every character in C programming is given an integer value to represent it. That integer value is known as ASCII value of that character.

For example: ASCII value of 'a' is 97.

14. How will you remove blank space from the given string?

```
#include <stdio.h>
int main()
{
    char text[100], blank[100];
    int c = 0, d = 0;
    printf("Enter some text\n");
    gets(text);
    while (text[c] != '\0')
    {
        if (!(text[c] == ' ' && text[c+1] == ' ')) {
            blank[d] = text[c];
            d++;
        }
        c++;
    }
    blank[d] = '\0';
    printf("Text after removing blanks\n%s\n", blank);
    return 0;
}
```

15. How will you remove digits from the given string?

```
#include<stdio.h>
```

```

int main(){
    char line[150];
    int i,j;
    printf("Enter a string: ");
    gets(line);
    for(i=0; line[i]!='\0'; ++i)
    {
        while (!((line[i]>='a'&&line[i]<='z') || (line[i]>='A'&&line[i]<='Z' ||
line[i]=='\0'))))
        {
            for(j=i; line[j]!='\0'; ++j)
            {
                line[j]=line[j+1];
            }
            line[j]='\0';
        }
    }
    printf("Output String: ");
    puts(line);
    return 0;
}

```

EX:NO:15	SUM OF AN INTEGER ARRAY USING POINTERS	DATE:
----------	--	-------

AIM :

To write a C program to find the sum of an integer array using pointers.

ALGORITHM:

Step 1 : Start

Step 2 : Declare the array a[10] and pointer variable p

Step 3 : Read Number of elements for array a[] and it is 'n'.

Step 4 : Using for loop, read array elements a[0,1,2.....,n-1] 'n' times

Step 5 : Assign Array a[] base address to pointer variable p i.e. p=a

Step 6 : Within for loop, do the following

i) Display value, address of array element using *(p+i), (p+i) respectively

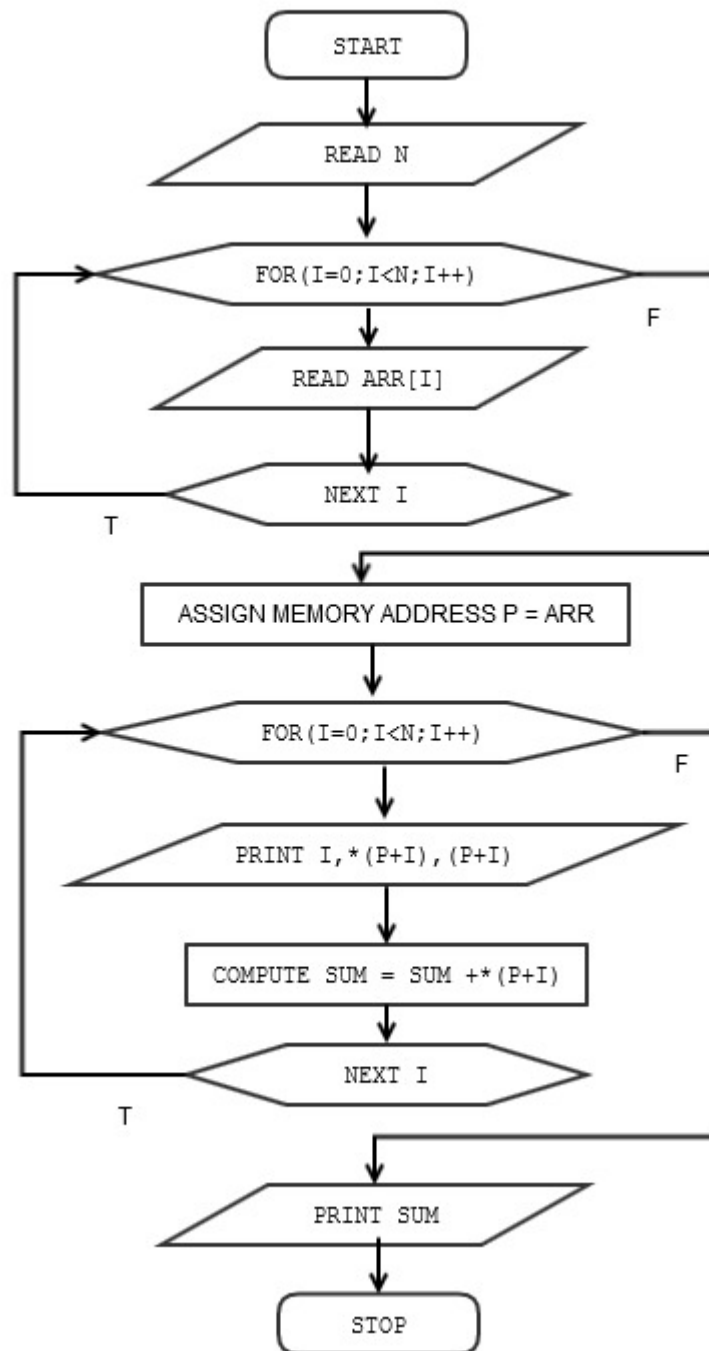
ii) Use Short hand operator to find sum .i.e. sum+= *(p+i)

Step 7 : Repeat Step 6 until the for loop get false

Step 8 : Print the sum value

Step 9 : Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.15 Sum Of Series Using Pointers
#include <stdio.h>

#include<conio.h>

void main()
{
    clrscr();

    int a[10],i,n,sum=0, *p;

    printf("\nSUM OF SERIES USING POINTERS");

    printf("\n*-*-*-*-*-*-*-*-*-*-*-*-*-*");

    printf("\nEnter the number of elements:");

    scanf("%d", &n);

    printf("\nEnter the Elements ");

    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    p=a;

    printf("\n-----");

    printf("\n\tARRAY\tVALUE\tADDRESS");

    printf("\n-----");

    for(i=0;i<n;i++)
    {

        printf("\n\t\ta[%d]\t%d\t%u", i, *(p+i), p+i);

        sum = sum + *(p+i);
```

```

    }

    printf("\n-----");

    printf("\nSum of all elements in array using pointer= %d", sum);

    getch();
}

```

OUTPUT:

SUM OF SERIES USING POINTERS

*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

Enter the number of elements:4

Enter the Elements 23 45 -78 120

ARRAY	VALUE	ADDRESS
a[0]	23	2293484
a[1]	45	2293488
a[2]	-78	2293492
a[3]	120	2293496

Sum of all elements in array using pointer = 110

RESULT:

Thus the program to find the sum of an integer array using pointers has been executed and verified successfully.

EX:NO:15 SUM OF AN INTEGER ARRAY USING POINTERS

1. What is the use of pointers?

- 1:pointers increase the execution speed
- 2:a pointers enable us to access a variable that is defined outside the function
- 3:pointers are more efficient in handling the data tables
- 4:pointers reduce the length and complexity of a program
- 5:the use of pointer array to character strings results in saving of data storage space in memory

2. What is pointer to an array?

Arrays are closely related to pointers in C programming but the important difference between them is that, a pointer variable can take different addresses as value whereas, in case of array it is fixed. This can be demonstrated by an example:

```
#include <stdio.h>
int main(){
    char c[4];
    int i;
    for(i=0;i<4;++i){
        printf("Address of c[%d]=%x\n",i,&c[i]);
    }
    return 0;
}
```

3. What is array of pointers ?

Just like array of integers or characters, there can be array of pointers too. An array of pointers can be declared as :

<type> *<name>[<number-of-elements>;

For example :char *ptr[3];

4. What is the use of for loop?

Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.

```
for(initialization;Test condition;increment or decrementation)
{
//statement block;
}
```

5. What is the difference between array and pointer?

A pointer is an address in memory where a variable is located. An array is a conceptual data representation consisting of a list of more than one item of a particular scalar type (int, float, char, structure, etc.) where each element is accessed by its index. The index can be thought of as a counter or enumerator telling you how many elements you have to skip over to get to the one you're interested in. Here's where addresses come in ... the location of a particular element in memory is offset from the so-called *base address* (i.e. the address of the starting element) by the value (sizeof(one element) * index #)

The C compiler is smart enough to take the sizeof() into account when you do pointer arithmetic, so you can find the address of the i-th element as (address of base) + i, rather than having to do the multiplication explicitly.

The idea of element address as offset also accounts for C's use of zero-relative array definitions. Since the first element is offset by 0 positions from the first element its index in C is 0, not 1. The second (ordinal) element is offset from the first by 1 position so its index is 1, and so on.

6. What is the difference between if and while statement?

if	while
(i) It is a conditional statement	(i) It is a loop control statement
(ii) If the condition is true, it executes some statements.	(ii) Executes the statements within the while block if the condition is true.
(iii) If the condition is false then it stops the execution the statements.	(iii) If the condition is false the control is transferred to the next statement of the loop

7. Construct an infinite loop using while?

```
while (1)
{
}
```

Here 1 is a non zero, value so the condition is always true. So it is an infinite loop.

8. What will happen when you access the array more than its dimension?

When you access the array more than its dimensions some garbage value is stored in the array.

9. Discuss about pointers and structures.

Pointer and structure can be used together in a program. Here arrow operator (->) is used. Arrow operator is also called member selection operator.

Consider the following declaration

```
struct shop  
{  
    char name[30];  
    int number;  
    float price;  
} product[2], *ptr;
```

This statement declares **product** as an array of two elements, each of type **struct shop** and **ptr** as a pointer to data objects of the type **struct shop**. The assignment,

ptr=product;

would assign the address of the **0th element of product to ptr**. That is the pointer **ptr** will now point to **product[0]**. Its members can be accessed using the following notation.

ptr→ name
ptr→ number
ptr→ price

The symbol → is called the arrow operator/member selection operator. **ptr**, is simply another way of writing **product[0]**. When the pointer **ptr** is incremented by one, it is made to point to the next record,

ie., **product[1]**

The following **for** statement will print the values of members of all the elements of **product** array.

```
    for(ptr=product;ptr<product+2;ptr++)  
    {  
        printf(“%s %d %f\n”, ptr→name, ptr→number,  
ptr→price);
```

}

10. Define Dynamic Memory Allocation (DMA).

This is a process of dynamically allocating memory to any variable that is being used in your C program. When the program is terminated, the dynamically allocated memory is deallocated. C provides **malloc()** and **calloc()** functions for dynamic memory allocation and the function **free()** for deallocation of memory. The prototypes are declared in **alloc.h** and **stdlib.h**.

11. What are the differences between malloc() and calloc()?

There are 2 differences.

First, is in the number of arguments. **malloc()** takes a single argument(memory required in bytes), while **calloc()** needs 2 arguments(number of variables to allocate memory, size in bytes of a single variable).

Secondly, **malloc()** does not initialize the memory allocated, while **calloc()** initializes the allocated memory to ZERO.

12. What is a null pointer?

There are times when it's necessary to have a pointer that doesn't point to anything. The macro **NULL**, defined in, has a value that's guaranteed to be different from any valid pointer. **NULL** is a literal zero, possibly cast to void* or char*.

Some people, notably C++ programmers, prefer to use 0 rather than NULL.

The null pointer is used in three ways:

- 1) To stop indirection in a recursive data structure
- 2) As an error value
- 3) As a sentinel value

13. What is a void pointer?

A void pointer is a C convention for "a raw address". The compiler has no idea what type of object a void Pointer "really points to." If you write

```
int *ip;
```

ip points to an int. If you write,

```
void *p;      p doesn't point to a void!
```

EX:NO:16	MAXIMUM ELEMENT IN AN INTEGER ARRAY USING POINTERS	DATE:
-----------------	---	--------------

AIM :

To write a C program to find the Maximum element in an integer array using pointers.

ALGORITHM:

Step 1 : Start

Step 2 : Declare the variable i,n,array[20],*maximum

Step 3 : Get the n value and store it an array

Step 4 : Within in for loop check $*(array+c) > *maximum$

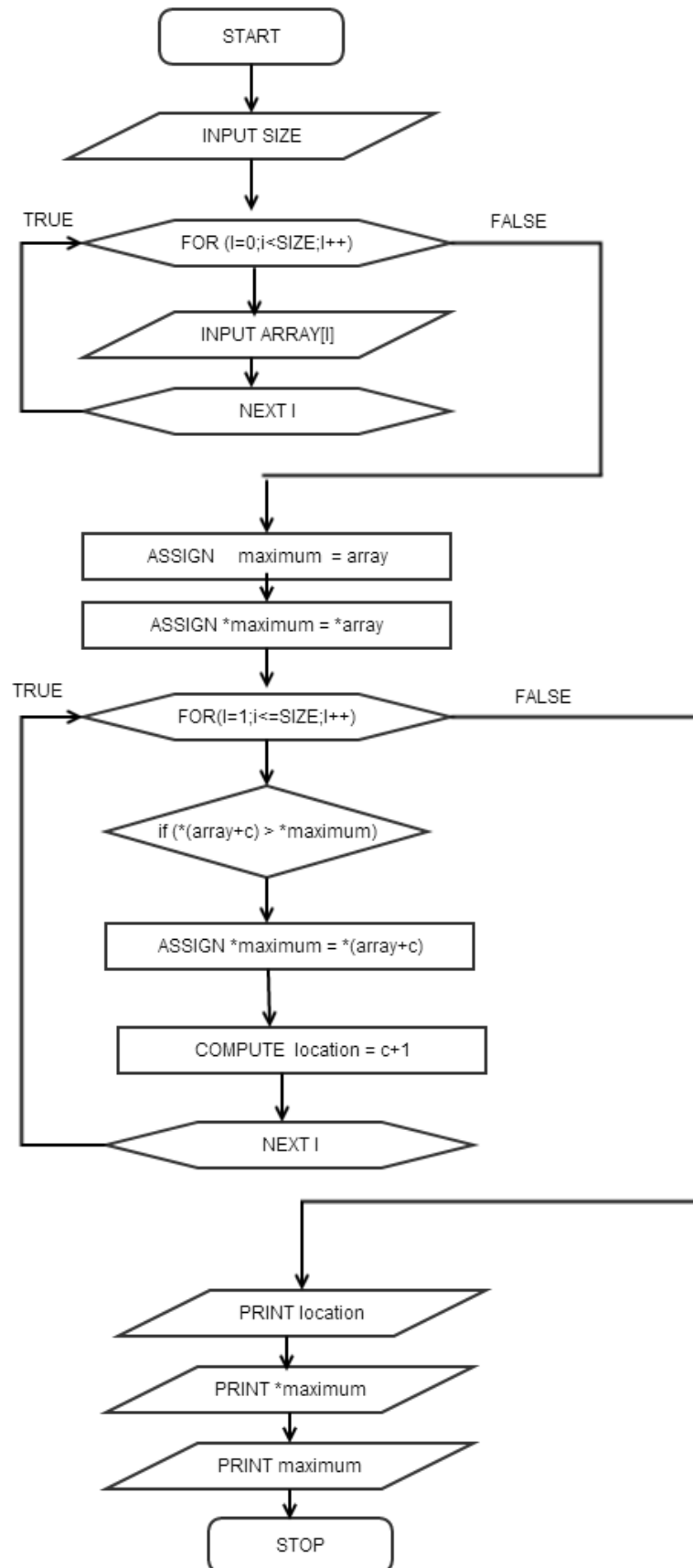
Step 5 : Assign $*maximum = *(array+c);$

Step 6: Repeat for loop until the condition gets false.

Step 6 : Print the value *maximum

Step 7 : Stop

FLOW CHART :



SOURCE CODE :

```
//Program No.16 MAX in an array using integer pointer
#include <stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    long array[100], *maximum, size, c, location = 1;
    printf("\nMAXIMUM IN AN ARRAY USING POINTER");
    printf("\n*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*");
    printf("\nEnter the number of elements in array : ");
    scanf("%ld", &size);
    printf("\nEnter %ld integers \n", size);
    for ( c = 0 ; c < size ; c++ )
        scanf("%ld", &array[c]);
    maximum = array;
    *maximum = *array;
    for (c = 1; c < size; c++)
    {
        if (*(array+c) > *maximum)
        {
            *maximum = *(array+c);
            location = c+1;
        }
    }
    printf("\nMaximum element found at location : %ld ",location);
    printf("\nAnd it's value is          : %ld",*maximum);
    printf("\nAnd it's Address is          : %u",maximum);
    getch();
}
```

OUTPUT :

```
MAXIMUM IN AN ARRAY USING INTEGER POINTER  
*_**_***_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*  
Enter the number of elements in array : 4  
Enter 4 integers  
567  
890  
123  
-78  
  
Maximum element found at location : 2  
And it's value is                  : 890  
And it's Address is                 : 2293124
```

RESULT:

Thus the program to find the Maximum element in an integer array using pointers

has been executed and verified successfully.

EX:NO:16 MAXIMUM ELEMENT IN AN INTEGER ARRAY USING POINTERS

1. Define Pointers.

A pointer is a variable which holds the address of other variable. Apart from the elements of basic data type pointers point to the different data items such as arrays, structures and functions that are used in the program.

The operators used with the pointers are the *address operator* (&) and the *dereferencing operator* (*). The address operator returns the memory location of a particular element and the dereferencing operator (**indirection operator**) specifies the value housed at that memory location.

2. What is a pointer variable?

A pointer variable is a variable that may contain the address of another variable or any valid address in the memory.

3. How to declare a pointer variable?

The syntax is **Data-type *pt-name;**

Here **pt-name** refers to valid pointer variable name. * is called as **indirection operator** also known as **dereferencing operator**. Data-type refers to a valid C data type. Pointer variable must be followed by '*' sign.

Example:

- **int *x;**
- **float *f;**
- **char *y;**

4. How are pointer variables initialized?

Initialization is the process of assigning the address of a variable to a pointer variable. Once a pointer variable has been declared we can use using '=' Assignment operator to initialize the variable.

For example:

int quantity;

int *p; /* Declaration*/

p=&quantity; /*Initialization*/

Pointer variable are initialized by one of the following two ways

- Static memory allocation
- Dynamic memory allocation

5. What is a pointer value and address?

A pointer value is a data object that refers to a memory location. Each memory location is numbered in the memory. The number attached to a memory location is called the address of the location.

6. What are the Features of Pointers?

- a. Pointers save memory space.
- b. Execution time with the pointer is faster because data is manipulated with the address, that is, direct access to memory location.

- c. The memory is accessed efficiently with the pointers. The pointer assigns the memory space and it also releases the memory. Dynamically memory is allocated.
- d. Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.

7. Difference between `const char* p` and `char const* p`.

In `const char* p`, the character pointed by 'p' is constant, so u cant change the value of character pointed by p but u can make 'p' refer to some other location. in `char const* p`, the ptr 'p' is constant not the character referenced by it, so u cant make 'p' to reference to any other location but u can change the value of the char pointed by 'p'.

8. What are the Advantages of Pointers?

- To point to different data structures
- Manipulation of data at different memory location is easier
- To achieve clarity and simplicity
- More compact and efficient coding
- To return multiple values via functions
- Dynamic Memory Allocation

9. Discuss about pointers & arrays.

When an array is declared, the compiler allocates a base address and sufficient amount of storage to contain all the elements of the array in contiguous memory locations. The base address is the location of the first element (index 0) of the array. Suppose we declare an array X as follows

`int x[5] = {1,2,3,4,5}`

Suppose the base address of x is 1000 and assuming that each integer requires two bytes, the five elements will be store as follows

<code>x[0]</code>	<code>x[1]</code>	<code>x[2]</code>	<code>x[3]</code>	<code>x[5]</code>
1	2	3	4	5
1000	1002	1004	1006	1008

The name **x** is defined as a constant pointer pointing to the first element, **x[0]** and therefore the value of **x** is 1000, the location where **x[0]** is stored.

For example:

x=&x[0]=1000

If we declare **p** as an integer pointer, then we can make the pointer **p** to point to the array **x** by the following assignment **p=x;**

This is equivalent to **p= &x[0];**

Now we can access every value of **x** using **p++** to move from one element to another.

The relationship between **p** and **x** is shown as

p=&x[0]=1000;

p+1=&x[1]=1002;

p+2=&x[2]=1004;

p+3=&x[3]=1006;

p+4=&x[4]=1008;

10. What do you mean by array of pointers?

Array of pointers contains collection of address. The address may be the address of variable or array elements. So far we have studied array of different standard data types such as array of int, float, character and so on.

In the same way the C language also supports array of pointers. It is nothing but a collection of addresses. Here we store addresses of variables for which we have to declare an array as a pointer.

11. Discuss about pointers & strings.

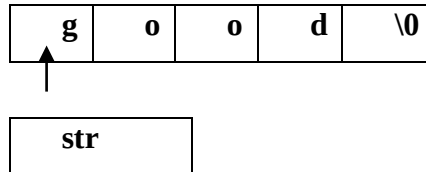
Strings are treated like character arrays and therefore, they are declared and initialized as follows, **char str[5]="good";**

The compiler automatically inserts **the null character '\0'** at the end of the string. C supports an alternative method to create strings using pointer variables of type **char**.

For example:

```
char *str="good";
```

This creates a string for the literal and then stores its address in the pointer variable **str**. The pointer **str** now points to the first character of the string “good” as:



We can also use the run-time assignment for giving values to a string pointer. For example,

```
char * string1;
```

```
string1="good";
```

12. What is pointer to a function?

A function like a variable has a type and an address location in the memory. It is therefore possible to declare a pointer to a function, which can be used as an argument in another function.

Pointer to a function contains the address of function in memory. Pointers are passed to a function as arguments. It helps to use it as argument in another function. The general form for declaring pointer to a function is

Syntax :

```
returntype (*fptr)();
```

where **fptr** is a pointer to a function. **fptr** returns the value of type return type.

13. Discuss about pointers and structures.

Pointer and structure can be used together in a program. Here arrow operator (**->**) is used. Arrow operator is also called member selection operator. Consider the following declaration

```
struct shop
{
char name[30];
int number;
```

```
float price;  
} product[2], *ptr;
```

This statement declares **product** as an array of two elements, each of type **struct shop** and **ptr** as a pointer to data objects of the type **struct shop**. The assignment,

```
ptr=product;
```

would assign the address of the **0th element of product to ptr**. That is the pointer **ptr** will now point to **product[0]**. Its members can be accessed using the following notation.

```
ptr→ name  
ptr→ number  
ptr→ price
```

The symbol **→** is called the arrow operator/member selection operator.

ptr, is simply another way of writing **product[0]**.

When the pointer **ptr** is incremented by one, it is made to point to the next record, ie., **product[1]**

The following **for** statement will print the values of members of all the elements of **product** array.

```
for(ptr=product;ptr<product+2;ptr++)  
{  
printf(“%s %d %f\n”, ptr→name, ptr→number, ptr→price);  
}
```

EX:NO:17	STUDENT DETAILS USING STRUCTURES	DATE:
----------	----------------------------------	-------

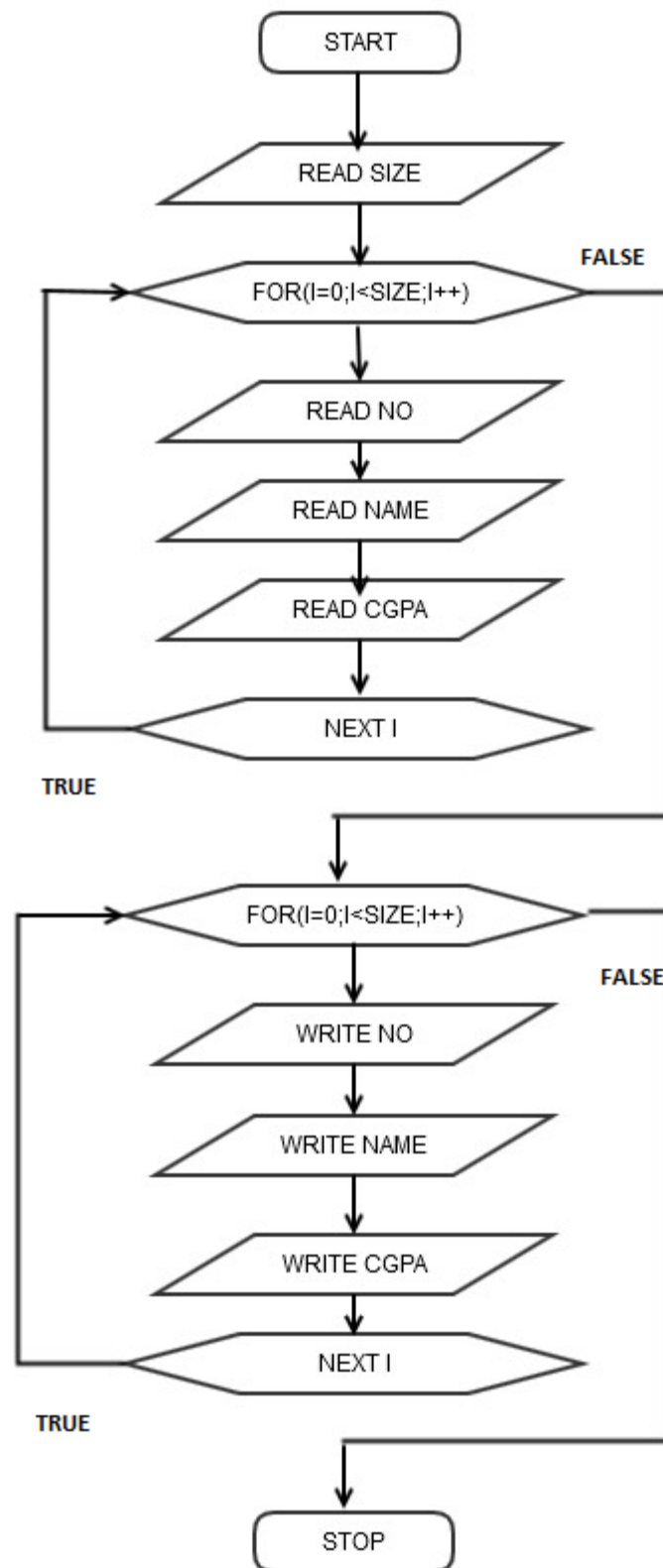
AIM :

To write a C program to create student details using Structures.

ALGORITHM:

- Step 1 : Start
- Step 2 : Declare rno, name[10], cgpa in structure name student
- Step 3 : Create structure variable as array i.e. .. s[10]
- Step 3 : Get the no.of students and it is 'n'
- Step 4 : Using for loop Access the roll no,name, cgpa for 'n' number of students
- Step 5 : Using for loop print the roll no, name, cgpa for all the students.
- Step 6 : Stop

FLOW CHART :



SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    struct student
    {
        int rno;
        char name[10];
        float CGPA;
        }s[10];
    int n,i;
    float dummy;
    printf("\n\n\t*****STUDENT NAME
    LIST*****\n\n");
    printf("\n Enter the Number of students:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        printf("\n Enter the roll number of student %d : ",i);
        scanf("%d",&s[i].rno);
        printf("\n Enter the name of student %d : ",i);
        scanf("%s",s[i].name);
        printf("\n Enter the CGPA of student %d : ",i);
        scanf("%f",&dummy);
        s[i].CGPA=dummy;
    }
    printf("\n\n The Student details are:\n");
    printf("-----\n\n");
    printf("ROLLNO \t\t Student Name \t\t CGPA\n");
    printf("*****\t\t*****\t\t*****\n\n");
    for(i=1;i<=n;i++)
    {
        printf("\n %d \t %s \t %.2f",s[i].rno,s[i].name,s[i].CGPA);
    }
    getch();
}
```

OUTPUT :-

```
*STUDENT INFORMATION USING STRUCTURES
*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*
Enter the Number of students:3

Enter the roll number of student 1 : 780

Enter the name of student 1 : Raman

Enter the CGPA of student 1 : 9.5

Enter the roll number of student 2 : 786

Enter the name of student 2 : Ravi

Enter the CGPA of student 2 : 8.4

Enter the roll number of student 3 :456

Enter the name of student 3 : Ashok

Enter the CGPA of student 3 : 7.7
```

The Student details are:

ROLLNO	Student Name	CGPA
780	Raman	9.50
786	Ravi	8.40
456	Ashok	7.70

RESULT:

Thus the program to create student details using structures has been executed and verified successfully.

PROGRAM : 18 STUDENT DETAILS USING STRUCTURES

1. How many types of data structures are there?

- a. Arrays
- b. Structures
- c. Linked List
- d. Trees
- e. Queues
- f. Stack
- g. File

2. What is a Structure?

A structure is an organized collection of different types of elements. A structure is also called as a record. Elements of a structure are called the members or fields. These fields may be of int, float, char or double. The **struct** is a keyword to provide a structure facility in C.

3. What do you mean by structure declaration?

Each and every structure must be defined and declared before it appears in C program.

The general form of structure definition and declaration is as follows.

```
struct name
{
    type1 data1;
    type2 data2;
    :
    typen datan;
};
```

Where,

struct - is a keyword
 name - name of the structure
 type1, type2...typen - are basic data types
 data1, data2...datan - are different data items or structure variables.
 And each is called a *member of a structure*.
 These are also called the *fields*.

The general form of declaring structure variables is

structure_type varlist;

where, structure_type – is the type of a structure. It takes the keyword **struct**
 followed by the name of the structure. It is a
derived data type.

varlist – one or more variables separated by commas.

4. What are embedded structures?

A structure within another structure is called an embedded structure.

Example:

```

struct salary
{
    char name;
    char department;
    struct
    {
        int dearness_allowance;
        int house_rent_allowance;
        int city_allowance;
    } allowance;
} employee;
  
```

This salary structure contains a member named allowance, which itself is structure with three members. The members contained in the inner structure namely dearness, house_rent and city can be referred as

employee.allowance.dearness
 employee.allowance.house_rent

employee.allowance.city

5. How do you access the members of a structure?

An element of a structure can be accessed by the **dot operator**. An **arrow operator** can also be used for accessing the structure members. To access a particular member, the dot operator must be placed between the name of the structure and the name of the structure member.

Syntax for accessing structure members is
structure_variable.member_name

6. What is the need for an array of structures?

Whenever the same structure is to be applied to a group of people, items or applications we will have to use an **array of structures**. In an array of structures each element is a structure itself.

Example:

struct class student [100]; defines an array called student that consists of 100 elements. Each element is defined to be of the type struct class.

7. How do you pass structure between functions?

C supports the passing of structure values as arguments to functions. There are three methods by which the values of structure can be transferred from one function to another function.

1. Pass each member of the structure as an actual argument of the function call.
2. Passing a copy of the entire structure to the called function.
3. Third approach employs a concept called pointers to pass the structure as an argument. In this case the address location of the structure is passed to the called function.

8. How user-defined data types are defined?

C provides, **typedef** (for **type definition**) statement that enables the programmer to define an alternate term to the basic data types. Once the **typedef** is made, the earlier datatype, all variables, arrays, structures etc., can be declared with the new data type.

Its syntax is

typedef old_data_type new_data_type;

where typedef - is a keyword

old_data_type - a basic data type

new_data_type- user-defined data type

9. Explain enumerated data type.

The **enum** is a keyword. It is used for declaring enumeration types. The programmer can create his/her own data type and define what values the variables of these data types can hold. This enumeration data type helps in reading the program.

10. What is Array of Structures?

Array of structures: an array having structure as its element is called an array of structures. that is, each element of the array represents a structure variable.

Syntax:- declaration

Structstructurename

{

.....

.....

} arrayname[size];

Example:

<pre>Struct student { char name[15]; int no; float perc; char grade; }s[3]={{"vasanth",20,80.5,'B'},{"Raju",15,70.1,' C'}, {"vini",25,91.1,'A'}};</pre>	<pre>for(i=0;i<3;i++) { Printf("\nName:%s",s[i].name); Printf("\nNo:%d",s[i].no); Printf("\nPercentage:%f",s[i].per c); Printf("\nGrade:%c",s[i].grade); } Return 0; }</pre>
--	---

<pre>int main() { inti;</pre>	
-------------------------------	--

11. Define nested Structure

NESTED STRUCTURE: Nesting of structures are nothing but structure within another structure (i.e., a structure can contain one or more structures as its members).

- A structure may be defined and/or declared inside another structure.

```
struct employee
{
    int empno;
    char name[15];
    struct employ_add
    {
        int no;
        char street[15];
        char area[15];
        long pincode;
    } address;
    Char deptname[15];
    Float salary;
} emp;
```

12. HOW TO PASS THE ADDRESS OF MEMBER TO FUNCTIONS ?

The member of a structure can also be passed to function by passing the address of the members.

In this, a method the address location of the members is passed to the called function, hence the address operator (&) is used before the structure name.

- The member of the structure can be passed to a function employ() as

```
employ(&emp.empno);  
employ(&emp.salary);  
employ(&emp.empname);
```

13. Define Union

Unions are same as structures except that they differ in storage. Unions have members of different data types. All the members of a union share the same storage area within the memory. At any time only one value can be stored.

A union is another compound data type like a structure that array hold objects of different types and sizes, with the compiler keeping back of the size.

Unions provide a way to manipulate different kinds of data in a single area of storage.

Give the syntax of union

Syntax

```
union <union name>  
{  
    data type member 1;  
    .  
    .  
    data type member N;  
} union_variable;
```

The syntax of a Union is identical to the of a structure except that the keyword struct is replaced with the keyword Union. However, there is a major distinction between is terms of storage.

Give an example for union

Example

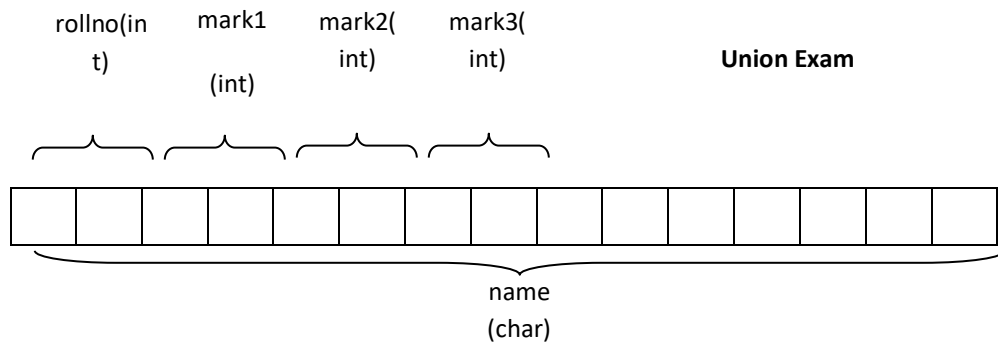
```
union exam  
{  
    introll_no;  
    char name[15];  
    int mark1,mark2,mark3;  
};
```

14. Explain how union variables are stored in memory

In the above example, Union exam has two members. The first member is a character array name having 15 characters (i.e., 15 Bytes).

The second member is of type `int` that requires 2 bytes for their storage. All the other members `mark1`, `mark2`, `amrk3` are of type `int`. Which require 2 bytes for their storage?

Memory Allocation



15. DIFFERENCE BETWEEN STRUCTURES AND UNIONS

S.No	Structures	Unions
1	Each member in a Structure occupies and uses its own memory space.	All the members of a Union use the same memory space.
2	The key word Struct tells us that we are dealing with structure.	The key word Union tells us that we are dealing with Unions.
3	All the members of a Structure can be initialized.	Only the first member of a Union can be initialized.
4	The members of a Structure are not stored in sequence of memory locations.	The members of an Union are stored in sequence of memory locations.
5	More memory space is required since each member is stored in a separate memory locations.	Less memory space is required since all members are stored in the same memory locations.

EX:NO:18	DISPLAY THE CONTENTS OF THE FILE ON THE MONITOR SCREEN	DATE:
----------	---	-------

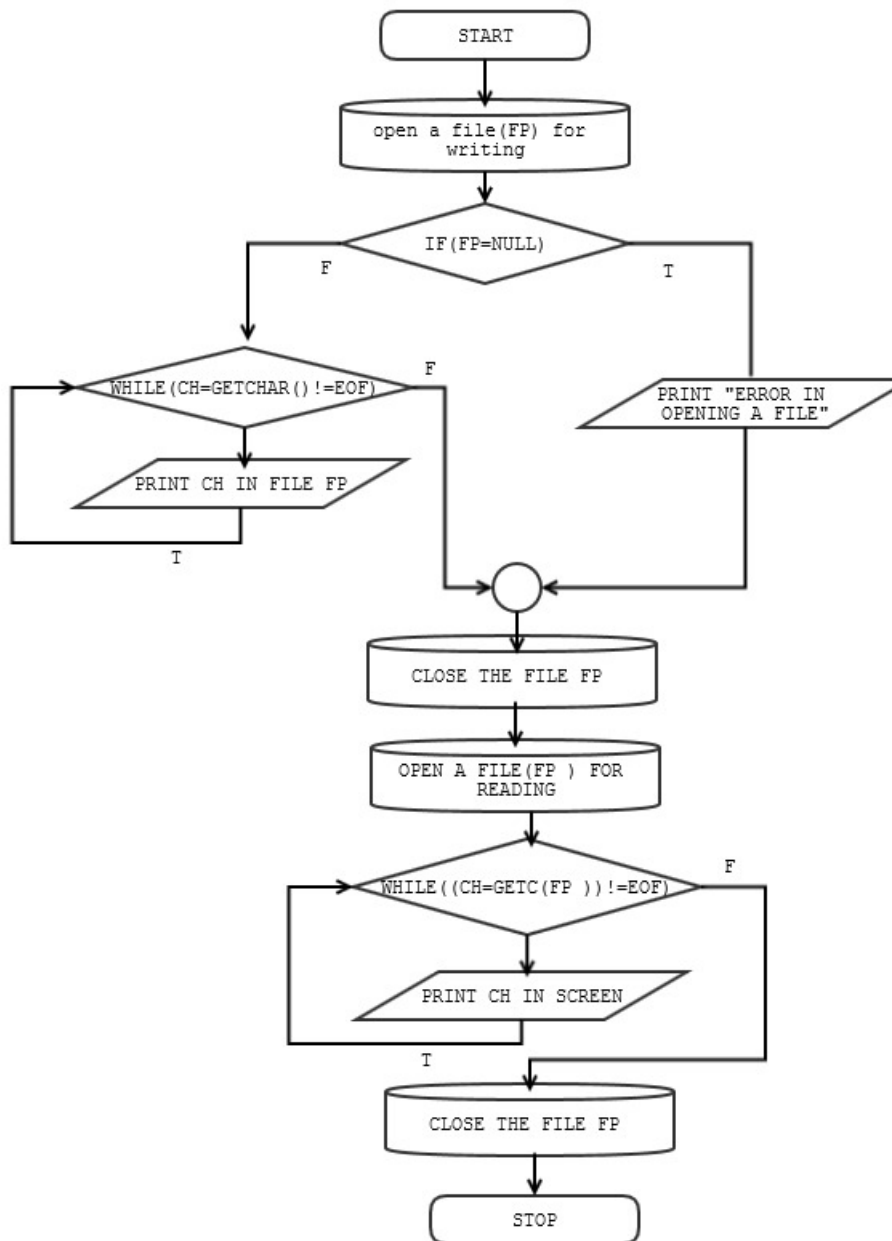
AIM :

To write a C program to display the contents of the file on the monitor screen.

ALGORITHM:

- Step 1 : Start
- Step 2 : Open the file textfile .txt in writing mode
- Step 3 : Check the condition if fp is null then print error
- Step 4 : Read the character
- Step 5 : Using while read the character up to EOF and print in fp
- Step 6 : Close the file
- Step 7 : Open the file textfile .txt in read mode
- Step 8 : Using while read the character up to EOF and print the character
- Step 9 : Close the file
- Step 10 : Stop

FLOW CHART :



SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
FILE *fp;
char ch;
printf("\n\tDISPLAY THE CONTENTS OF THE FILE ON
THE MONITOR SCREEN");
printf("\n\t*-**-*_*-**-**_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*_--*\n");
fp=fopen("textfile.txt","w");
if(fp==NULL)
printf("\nError in file opening");
else
{
printf("\nEnter characters [press ctrl+z to exit:]");
while((ch=getchar())!=EOF)
putc(ch,fp);
}
printf("\nFile Content saved in textfile.txt");
fclose(fp);
fp=fopen("textfile.txt","r");
printf("\n The content read from file and display on the screen");
printf("\n ----- \n");
while((ch=getc(fp))!=EOF)
printf("%c",ch);
fclose(fp);
getch();
}
```

OUTPUT:

```

DISPLAY THE CONTENTS OF THE FILE ON THE MONITOR SCREEN
*_**_**_**_**_**_**_**_**_**_**_**_**_**_**_**_**_**_*
Enter characters [press ctrl+z to exit]:The Expansion of computer is
Commonly Operated Machine Particulary Used for Technology and
Educational Research^Z
^Z
File Content saved in textfile.txt
  The content read from file and display on the screen
-----
The Expansion of computer is Commonly Operated Machine Particulary
Used for Technology and Educational Research

```

RESULT:

Thus the program to display the contents of the file on the monitor screen has been executed and verified successfully.

EX: NO: 18 DISPLAY THE CONTENTS OF THE FILE ON THE MONITOR SCREEN

1. What is a file?

Group of related record form a **FILE**. FILE is a place on the disk where group of related data is stored. DATA FILE is used to store information in floppy disk, hard disk. Field is a data item used to store a single unit. Group of related field constitute a RECORD. FILE is classified into three types. They are

Numeric file - It contains numeric data.

Text file - It consists of text data.

Data files -It consists of data grouped in the form of structure. It may have text, numeric data or both.

2. Classify file operation levels.

There are two FILE operation levels. They are

3. Low-level Input/Output operations

It uses UNIX system calls.

4. High-level Input/Output operations

It uses functions in standard C Input Output Library.

Function Name	Operations
fopen()	Creates a new file for use. Opens an existing file for use.
fclose()	Closes a file which has been opened for use.
fgetc()	Reads a character from a file.
fputc()	Writes a character to the file.
fprintf()	Writes set of data values to a file.
fscanf()	Reads a set of data values from a file.
getw()	Reads an integer from a file.
putw()	Writes an integer to a file.
fseek()	Sets the position to desired position in the file.
ftell()	Gives the current position in the file.
rewind()	Sets the position to the beginning of the file
fgets()	Reads a string from the file
fputs()	Writes a string to the file.
fread()	Reads unformatted data from the file
fwrite()	Writes unformatted data to the file

To store data in a file Secondary Memory is used. When a file is retrieved from an Operating System OS specifies the filename, Data structures and Purpose. Here File name is a string of characters. It contains two parts. First one is Primary name and then the other is optional period with extension.

5. What is a data file?

A *data file* (or simply file) is a collection of logically related information.

FILE is a data structure used to declare a pointer to a data file.

6. Why data files are required?

Data files can be accessed for the purpose of reading data from them and writing data back into them. The data read from the files are assigned to the variables in the

program and similarly, the values of variables from the source program are written into the files.

7. Classify the File types?

There are two types of files 1) Sequential file 2) Random accesses file.

- **Sequential File:** In this type data are kept sequentially. If we want to read the last record of the file we need to read all the records before reading that record. It takes more time.
- **Random Access File:** In this type data can be read and modified randomly. That is, if we want to read the last record of the file, we can read it directly. It takes less time as compared to a sequential file.

8. Why FILE data type is required?

FILE is a structure data type. It is used to establish file buffer area.

To get buffer area the syntax is **FILE *fp;**

where **fp** is a file pointer or stream pointer. **fp** contains all the information about file. It serves as link between system and program. **FILE** is a keyword. It refers to the file control structure for streams.

9. List the basic file operations that c supports.

Like most other languages, C supports a number of functions that have the ability to perform basic file operations, which include:

1. Naming a file
2. Opening a file
3. Reading data from a file.
4. Writing data to a file, and
5. Closing a file.

10. What are the steps involved for File operations?

There are three steps for file operation in C.

- a. Opening a file
- b. Reading or writing a file.
- c. Closing a file.

11. Explain the syntax for opening a file.

“**fopen**” function is used to open a stream or data file. File should be opened before reading or writing from it. If a file is opened ‘fopen’ functions returns FILE pointer to the beginning of buffer area. If file is not opened **NULL** value is returned.

The Syntax of fopen is, **FILE *fopen(“filename”, “mode”);**

fopen() function returns pointer to structure associated with the file. fopen() function needs two arguments of type string. First argument **filename** refers to the name of file which is to be opened. Second argument **mode** refers to file mode.

12. Explain the syntax for closing a file.

File is closed when all input output operations are completed. **fclose()** function is used to close opened file. This function makes all buffers associated with the file empty and all links to the file broken. Whenever we want to reopen the same file we have to close all other file.

The syntax for closing a file is **fclose(fp);**

EX:NO:19	GET INPUT FROM THE KEYBOARD AND RETRIEVE FROM THE OUTPUT FILE	DATE:
----------	--	-------

AIM :

To write a C program to Create a File by getting the input from the keyboard and retrieve the contents of the file using file operation commands.

ALGORITHM:

- Step 1 : Start
- Step 2 : Open the file itemdet.dat in writing mode
- Step 3 : Using if condition check (fp==NULL) then error
- Step 4 : Read itemnumber itemname Price Quantity from the keyboard using for loop
- Step 5 : Write itemnumber itemname Price Quantity to the file

Step 6 : Close the file

Step 7 : Open the file itemdet.dat in reading mode

Step 8 : Using for loop, Access the file records one by one and Process it
by following steps

A) Calculate $\text{Net} = \text{Price} * \text{Quantity}$

B) Calculate $\text{Overall} += \text{Net}$

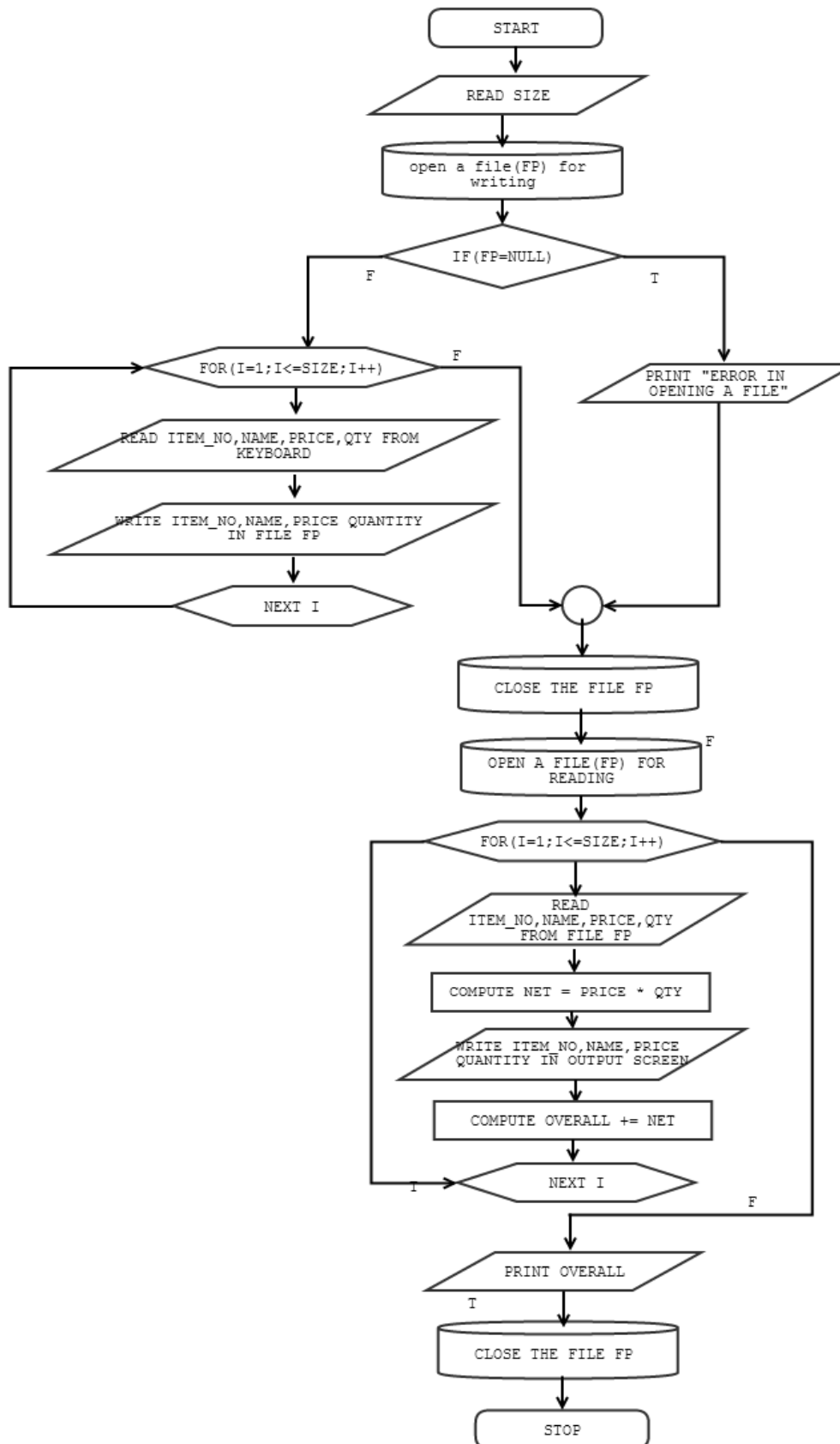
C) Print itemnumber, itemname, Price, Quantity,
netprice, from the file to output screen

Step 9 : Display overall value.

Step 10 : Close the file

Step 11 : Stop

FLOW CHART:



```
#include <stdio.h>
#include <stdlib.h>
#include<conio.h>
void main()
{
clrscr();
FILE *fp;
char item[30];
int i,no,size;
float price,qty,net,overall;
printf("\n\tDISPLAY THE CONTENTS OF THE FILE USING FILE
OPERATION COMMANDS");
printf("\n\t*****");
printf("\nEnter the no.of records : ");
scanf("%d",&size);
fp=fopen("itemdet.txt","w");
if(fp==NULL)
{
printf("\nCannot open file.\n");
exit(1);
}
for(i=1;i<=size;i++)
{
printf("\nEnter the record %d details",i);
printf("\n-----");
printf("\nItem No      :"); scanf("%d",&no);
printf("\nItem Name    :"); scanf("%s",&item);
printf("\nItem Price   :"); scanf("%f",&price);
printf("\nItem Quantity :"); scanf("%f",&qty);
fprintf(fp, "\n%6d\t%6s\t%8.2f\t%8.2f",no,item,price,qty); /* write to
file */
}
fclose(fp);
fp=fopen("itemdet.txt","r");
printf("\n-----");
printf("\nItemnumber\tItemname\tPrice\tQuantity\tNetprice");
printf("\n-----");
```


Itemnumber netprice	Itemname	Price	Quantity

-			
45	scale30cm	10.00	50.00
500.00			
89	StickPen	8.00	35.00
280.00			
9	InkPen	34.50	25.00
862.50			

-			
		Net Total :	
1642.50			

-			
RESULT:			

Thus the program to Create a File by getting the input from the keyboard and retrieve the contents of the file using file operation commands has been executed successfully.

EX:NO:19 GET INPUT FROM THE KEYBOARD AND RETRIEVE FROM THE OUTPUT

1. Define File?

Group of related record form a **FILE**. FILE is a place on the disk where group of related data is stored. DATA FILE is used to store information in floppy disk, hard disk. Field is a data item used to store a single unit. Group of related field constitute a RECORD. FILE is classified into three types. They are

- **Numeric file**

It contains numeric data.

- **Text file**

It consists of text data.

- **Data files**

It consists of data grouped in the form of structure. It may have text, numeric data or both.

2. Define Record?

A **record** is the collection of fields that relate to a single entity. For example, we could have a student record that includes fields for the student's name, address, homeroom, date of birth, etc. A product record could include fields for the serial number, description, cost price, quantity in stock, etc.

3. Define field?

A **field** is an item of stored data. A field could be a name, a date, an address, a description, a quantity, etc. When a field is defined it is given a name (identifier) and a type, that defines the type of data that will be stored in that field

4. Define Database?

A database is an application that manages data and allows fast storage and retrieval of that data.

5. What are the different modes of file?

File Modes:

Mode specifies the purpose of opening a file. The following table shows different file modes and their purpose

Mode	Description
"r"	Read the content of the file
"w"	Write content into the file.
"a"	Append the content to the existing file.
"r+"	Read and Write
"w+"	Write and Read
"a+"	Append and Read

6. Write the Syntax of opening a file.

fopen()-create a new file or opens an existing file for use.

Syntax : `fp=fopen("filename","mode");`

Example : `fp1=fopen("stud.txt","w");`

7. What are the functions used for I/O errors

- a. `feof()` function
- b. `ferror()` function

8. What is the use of `feof()` function

It is used to test for an end of file condition.

9. What is the use of `rewind()` function

It resets the Position of the file pointer to the start of the file. It is used to read a file more than once without having to close and open the file.

10. What is `fscanf` and `fprintf()` function

fscanf()	fprintf()
It is used to read the data items from the file	It writes the mixed data items into a file
Syntax:- <code>fscanf(fp,"control string",list);</code> eg: <code>fscanf(fp1,"%s%d",name,&no);</code>	Syntax:- <code>fprintf(fp,"control string",list);</code> eg: <code>fprintf(fp1,"%s%d",name,no);</code>

11. Define the use of `getw()` and `putw()` function

getw()	putw()
It is used to read an integer number from file.	It is used to write an integer number into a file
Syntax: <code>n=getw(fp);</code> Eg: <code>i=getw(fp1);</code>	Syntax: <code>putw(n,fp);</code> Eg : <code>putw(l,fp1);</code>

12. Differentiate `getc()` and `putc()` functions

getc()	putc()
It is used to read one character at a time from a file	It is used to write one character into the file.

Syntax: <code>c=getc(fp);</code> Eg: <code>m=getc(fp1);</code>	Syntax : <code>putc(fp);</code> Eg: <code>putc(fp1);</code>
--	---

13. What is random access to a file?

C provides a set of functions for writing or reading data to or from the data file randomly.

These functions are given below,

- fseek ()** - Repositions the file pointer of a stream
- ftell ()** - Returns the current file pointer
- rewind ()** - takes the control back to the beginning of the file

EX:NO:20	PASSING OF PARAMETER USING COMMAND LINE	DATE:
----------	---	-------

AIM :

To write a C program to pass the parameter using command line arguments.

ALGORITHM:

Step 1 : Start

Step 2 : Declare argument count and argument vector in the main function

Step 3 : Using if condition check ($\text{argc} < 3$) ,if it is true then quit the process

Step 4 : Otherwise, use for loop and do the following for argc times.

a).sum the argument vector values and it is done by $\text{Sum} + \text{atoi}(\text{argv}[\text{i}])$

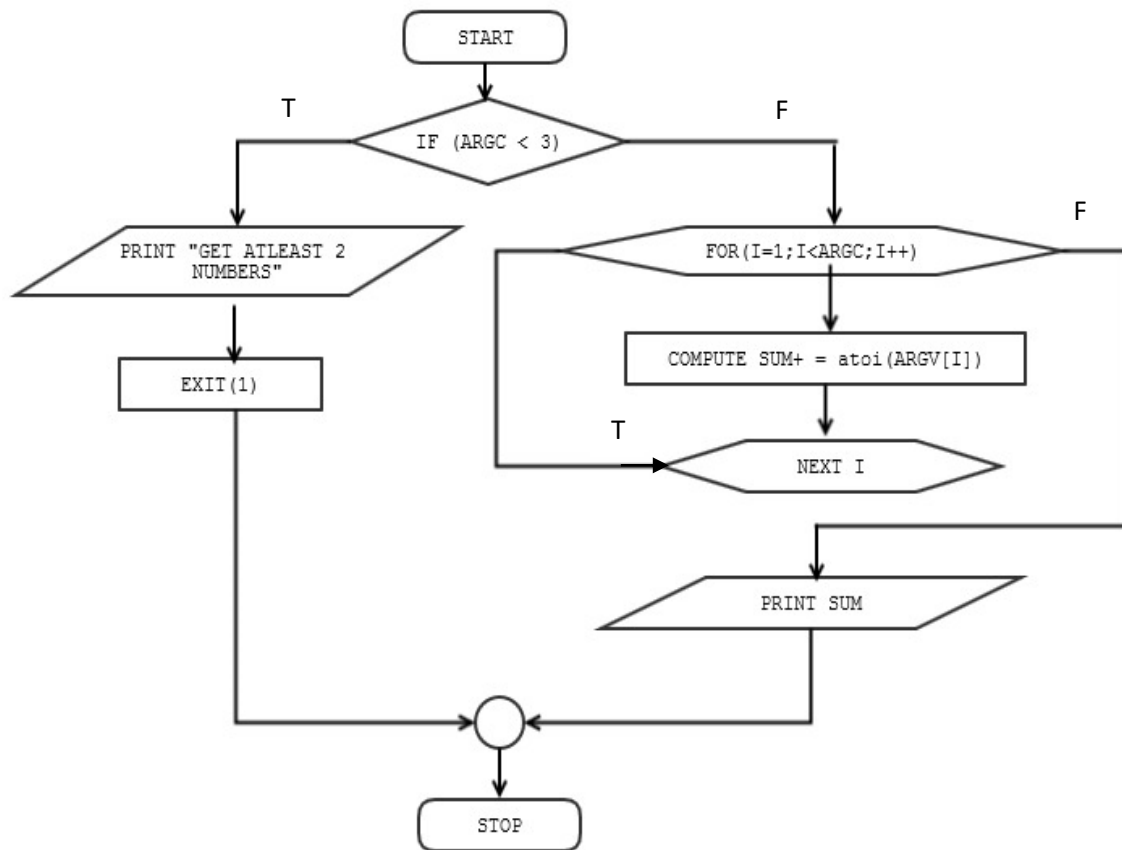
b).The convention function, $\text{atoi}()$ is used to convert the command line

string argument to number.

Step 5 : Print the sum value

Step 6 : Stop

FLOW CHART :



SOURCE CODE :

```

#include<stdio.h>
#include<stdlib.h>
#include<stdio.h>
void main(int argc , char *argv[])
{
    clrscr();
    int i,sum=0;
    printf("\n\tSUM OF INTEGERS USING COMMAND LINE
    ARGUMENT");
    printf("\n\t*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*");
    if(argc<3)
    {
        printf("\nAtleast Type 2 Numbers");
        exit(1);
    }
    printf("\nThe sum is : ");
    for(i=1;i<argc;i++)
  
```

```

        sum = sum + atoi(argv[i]);
    printf("%d",sum);
    getch();
}

```

OUTPUT :

```

GOTO -> START -> RUN -> CMD ->GOTO PROGRAM FOLDER THEN TYPE

D:\CP 2014\CP PRGMS>SUM_CMD_LINE.EXE 15 90 78 45 32 -24

        SUM OF INTEGERS USING COMMAND LINE ARGUMENT
        *-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
The sum is : 236

D:\CP 2014\CP PRGMS>

```

RESULT:

Thus the given program to pass the parameter using command line arguments has been executed successfully.

EX:NO:20 PASSING OF PARAMETER USING COMMAND LINE ARGUMENTS

1. Explain command line arguments?

An executable program that performs a specific task for operating system is called as command. The commands are issued from the prompt of operating system. Some arguments are associated with the commands; hence these arguments are called as command line arguments.

To access command line arguments, declare the main function and parameter as,

```
main(int argc, char *argv[ ])  
{  
    .....  
}
```

2. What is the use of argc()

It is an argument counter. It is used to pass the number of arguments in the command line to the main() function of the program.

3. What is the use of argv()

It is an argument vector. It is used to represent an array of character pointers that points to the command line arguments.

4. Define “C” Preprocessor.

This is *macroprocessor* program that processes the source program before it is compiled. It is a collection of special statements called ***preprocessor statements*** or ***preprocessor directives***. These directives are executed before the C program passes through the compiler.

5. What are the rules for defining preprocessor?

- All preprocessor directives begin with sharp sign or hash symbol. i.e., (#)
- It starts in first column.
- It should not terminate with semicolon.
- It may appear at any place in the source file. i.e., Outside the function, inside the function or
inside compound statements.

6. What are the Preprocessor Directives?

These are statements which begin with the # symbol placed in the first column. They are placed before the function **main()**. There will be no semicolon at the end of such statements.

There are three types of preprocessor directives. They are,

1. Macro substitution directives
2. File inclusion directives

3. Conditional compilation directives

7. Define Macro Substitutions.

The Macro substitution is a process of replacing an identifier of a C program by a constant or a symbolic constant. This can be accomplished by the directive **#define**. The **#define** is a macro definition statement.

Its **syntax** is as follows:

#define identifier CSCE

where, **#define** - a hash define directive

identifier - a valid C identifier (upper case)

CSCE - may be Constant, Symbolic Constant or Expression

Where, **identifier** also known as “**macro name**” or “**macro template**”.

Eg.,

```
#define FIVE 5
```

```
#define SIX FIVE+1
```

```
#define SEVEN SIX+1
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
{
```

```
    printf(“%d \t %d \t %d”,FIVE,SIX,SEVEN);
```

```
    getch();
```

```
}
```

8. State any two advantages of macro definition

1. It helps in generation of faster and compact code
2. It saves programmer's Time

9. Define File inclusion directives.

File inclusion is a process of inserting the external file containing the desired function definition into the program. The syntax of **#include** is as follows:

#include filename

where,

#include - is a preprocessor directive for the file inclusion

filename - is the name of the file containing the required function definition

#include<filename> – It make preprocessor to look for file with specified filename in special directories set aside for files.

#include“filename” – It make preprocessor to look for file with specified filename in Current working directory.

10. Define Conditional compilation directives.

Conditional compilation is a process of selecting a particular segment of code for compilation depending on the condition. The **#if, #else, #elif, #endif, #ifdef, #ifndef** directives are used for conditional compilation of the program.

The general form is,

```
#if constant expression
    sequence of statements
#else
    sequence of statements
#endif
```

Example:

```
main()
{
#ifdef INTEL
    code suitable for a Intel PC
#else
    code suitable for a Motorola PC
#endif
    code common to both the computers
}
```

11. Define delimiters in ‘C’.

Symbol	Delimiters	Use
:	Colon	Useful for label
;	Semicolon	Terminates Statement
()	Parenthesis	Used in expression and functions

[]	Square Bracket	Used for array declaration
{ }	Curly Brace	Scope of statement
#	Hash	Preprocessor directive
,	Comma	Variable Separator